

國立陽明交通大學
資訊科學與工程研究所
碩士論文

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University
Master's Thesis

IntenSensePot: 微調 LLM 在蜜罐中實現意圖感知的主
動交戰策略

IntenSensePot: Finetuned LLMs for Intent-aware
Engagement in Honeypots

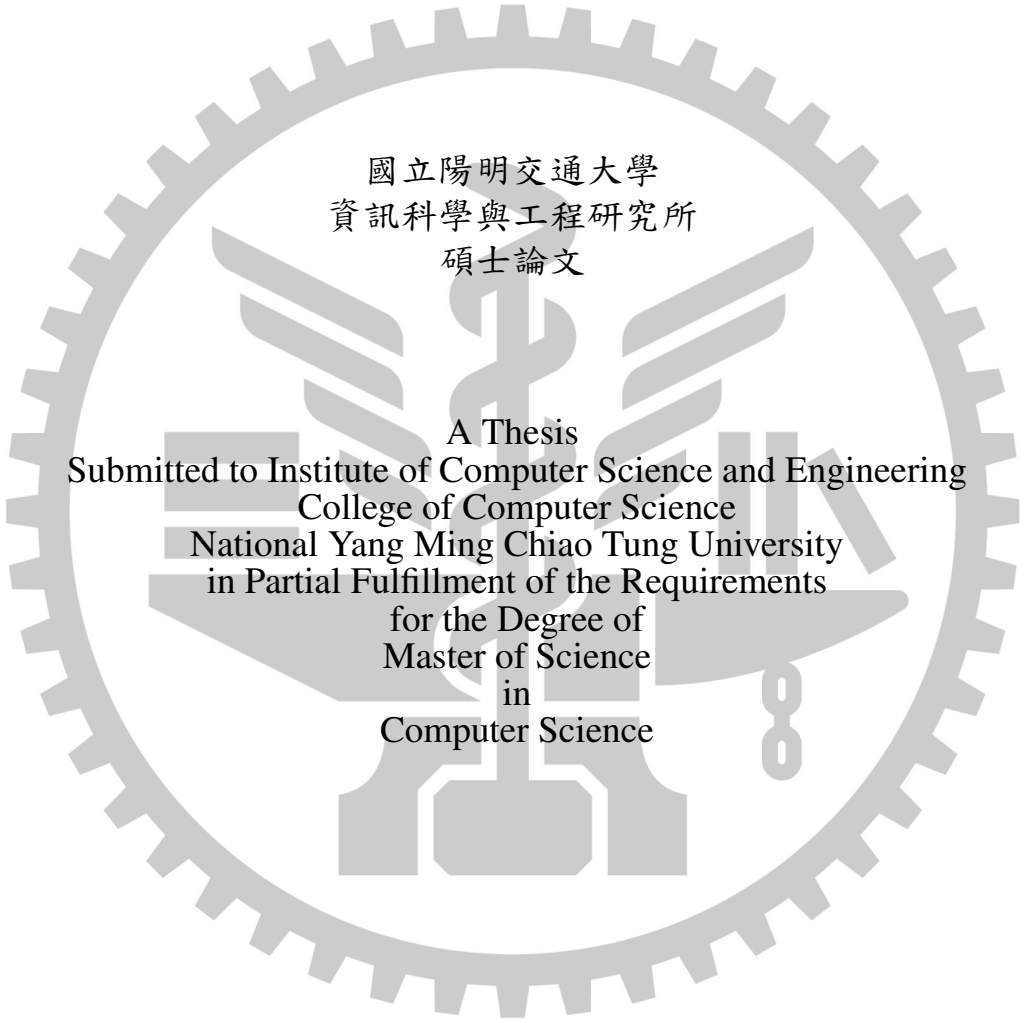
研 究 生：施儒宏 (Shih, Ru-Hung)
指導教授：林盈達 (Lin, Ying-Dar)

中華民國 一一四年六月
June 2025

IntenSensePot: 微調 LLM 在蜜罐中實現意圖感知的主動交戰策略
IntenSensePot: Finetuned LLMs for Intent-aware Engagement in
Honeypots

研 究 生：施儒宏
指導教授：林盈達 博士

Student：Ru-Hung Shih
Advisor：Dr. Ying-Dar Lin



國立陽明交通大學
資訊科學與工程研究所
碩士論文

A Thesis
Submitted to Institute of Computer Science and Engineering
College of Computer Science
National Yang Ming Chiao Tung University
in Partial Fulfillment of the Requirements
for the Degree of
Master of Science
in
Computer Science

June 2025
Taiwan, Republic of China

中華民國 一一四年六月

誌 謝

施儒宏 於

國立陽明交通大學 資訊科學與工程研究所

中華民國 一〇四年六月



IntenSensePot: 微調 LLM 在蜜罐中實現意圖感知的主動交戰策略

學生：施儒宏

指導教授：林盈達 博士

國立陽明交通大學
資訊科學與工程研究所

摘 要

隨著自動化攻擊工具的快速演進，蜜罐（Honeypot）作為主動式防禦系統，需維持高度的真實性與動態性，以有效誘使攻擊者並收集威脅情資。近期研究嘗試引入強化學習（RL）來提升蜜罐的動態回應能力，但此類方法受限於單指令單次回應的框架，無法有效識別跨指令的攻擊意圖並動態部署誘餌，導致錯失獲取攻擊樣本的機會。

為解決此問題，本研究提出一個結合大型語言模型（LLM）與 MITRE ATT&CK、MITRE Engage 框架的新型蜜罐系統。我們透過兩階段微調方法提升模型的意圖感知與交戰策略選擇能力。首先，我們基於 ATT&CK 紅方意圖與真實工具，並利用商用模型根據攻擊者意圖標註思考鏈，將商用模型的推理能力蒸餾至本地 LLM 中。其次，我們採用強化微調（GRPO）來修正 LLM 的交戰策略。為克服長期交戰中獎勵稀疏導致模型難以收斂的問題，我們使用獎勵模型，從交戰結果學習指令序列與策略間的獎勵關係，透過相對優勢引導 LLM 偏移策略偏好並適應蜜罐環境。

實驗結果顯示，在偵測戰術意圖的 F1-score 上能夠從 0.29 提升至 0.84 外，透過消融研究，該方法亦提升模型在選擇策略時和攻擊者意圖的一致性，由 0.54 提升至 0.78。此外，在實際交戰效能的評估中，本模型亦展現出極大的優勢。首先，在交戰深度方面，本模型藉由維持高度的系統擬真性並提供具深度的動態誘餌，將最大交戰深度提升至 5.92，相較於 QRASSH（1.32）大幅提升 348%，相較於 LLM Honeypot（3.00）亦提升 97%。其次，在交戰多樣性方面，本模型達到 8.96 的相異技術數，相較於 QRASSH（3.66）提升 144%，相較於 LLM Honeypot（5.10）提升 75%。

關鍵字：高互動蜜罐, MITRE ATT&CK, MITRE Engage, 大型語言模型, 意圖感知交戰, 監督微調, 強化微調

IntenSensePot: Finetuned LLMs for Intent-aware Engagement in Honeypots

Student : Ru-Hung Shih Advisor: Dr. Ying-Dar Lin

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

Abstract

As automated attack tools evolve, honeypots must maintain high authenticity and dynamism to deceive attackers and collect threat intelligence. Recent research applying reinforcement learning (RL) to enhance honeypot responses are limited by single-command, single-response frameworks. They fail to recognize cross-command attack intents or dynamically deploy decoys, missing opportunities to capture valuable attack samples.

To address this, we propose an LLM-based honeypot integrating MITRE ATT&CK and Engage frameworks via a two-stage fine-tuning approach. First, we distill reasoning capabilities from commercial models into a local LLM using Chain-of-Thought (CoT) annotations grounded in attacker intents derived from ATT&CK. Second, we apply Group Relative Policy Optimization (GRPO) to optimize engagement strategies. To mitigate sparse rewards in long-term interactions, a proxy reward model maps command sequences to strategies, directly guiding policy alignment via relative advantage.

Experiments show that our method improves tactical intent detection F1-score from 0.29 to 0.84. Furthermore, ablation studies confirm that our method enhances the alignment between the model's strategy selection and the attacker's intent, raising it from 0.54 to 0.78. In practical engagement evaluations, our model also exhibits significant advantages. First, in terms of engagement depth, by maintaining high system fidelity and providing dynamic decoys with depth, our model achieves a maximum engagement depth of 5.92, a 348% improvement over QRASSH (1.32) and a 97% improvement over LLM Honeypot (3.00). Second, in terms of engagement diversity, our approach reaches 8.96 unique techniques, representing a 144% improvement over QRASSH (3.66) and a 75% improvement over LLM Honeypot (5.10).

Keywords: High-interaction Honeypots, MITRE ATT&CK, MITRE Engage, LLM, Intent-awareness Engagement, Supervised Finetuning, Reinforcement Finetuning

Contents

Chinese Abstract	i
English Abstract.....	ii
Contents	iii
List of Figures	v
List of Tables.....	vi
1 Introduction.....	1
2 Background & Related Work.....	4
2.1 Honeypot.....	4
2.2 Reinforcement Learning in Honeypot	5
2.3 MITRE Framework.....	6
2.4 Group Relative Policy Optimization.....	8
2.5 Related Work	9
3 Problem formulation.....	12
3.1 Notation Table.....	12
3.2 Problem Statement.....	15
4 Solution Approach	18
4.1 Solution Overview	18
4.2 Supervised Fine-Tuning for Intent-Aware Chain-of-Thought	19

4.3	Action Space	21
4.3.1	Reward Design	23
4.4	Reinforcement Fine-Tuning for Engagement Intuition	25
5	Implementation	27
5.1	System Deployment and Honeypot Architecture.....	27
5.2	Red Team Agent and Trajectory Collection	29
5.3	SFT Dataset Construction and Training.....	32
5.4	Reward Modeling and GRPO Training.....	34
6	Experiment Results	37
6.1	Evaluation setup.....	37
6.2	Engagement performance	38
6.3	Intent Detection and Intent-Strategy Alignment.....	40
6.4	Reward Model and GRPO Training.....	43
6.5	Discussion and Limitation	46
7	Conclusions and Future works.....	47
7.1	Conclusions.....	47
7.2	Future Works.....	48
	References.....	49
	Appendix 附錄.....	54

List of Figures

Figure 1	MITRE ATT&CK Enterprise Matrix	6
Figure 2	MITRE Engage Matrix	7
Figure 3	Reinforcement Finetuning - GRPO	9
Figure 4	Solution Framework	18
Figure 5	Construction of Chain of Thought (CoT)	21
Figure 6	Action inference pipeline	23
Figure 7	Reinforcement Fine-tuning Pipeline	26
Figure 8	Honeypot architecture	29
Figure 9	Red Team Agent Structural Output	29
Figure 10	SFT and RFT Training pairs	33
Figure 11	Max attack depth over 50 engagement sessions.	38
Figure 12	Unique tactic counts over 50 engagement sessions.	39
Figure 13	Detection accuracy comparison across different methods.	41
Figure 14	Strategy and deployment relevance under intent-conditioned (M0) and intent-free (M1) generation, for both base and fine-tuned models.	42
Figure 15	Per-primitive predicted reward and training loss across training check- points.	44
Figure 16	Training reward and strategy probability drift during GRPO training. . .	45

List of Tables

Table 1	Comparison of Adaptive Honeypot systems	10
Table 2	Notations	14
Table 3	Action primitives and their corresponding MITRE Engage activities.	22
Table 4	Attack Stages	24
Table 5	MCP Tool Specification for Honeypot Strategy Primitives	28
Table 6	Supervised finetuning configuration	32
Table 7	Reward model configuration	34
Table 8	Reinforcement finetuning configuration	36
Table 9	Baselines in experiments. Llama3.1 refers to Llama-3.1-8B-Instruct.	37

Chapter 1. Introduction

As cyberattacks become increasingly automated, honeypots must exhibit both high realism and dynamic interactivity to capture unknown threat intelligence. Early simulation-based honeypots, including COWRIE[1] and recent LLM-driven simulators [2, 3, 4], primarily focus on generating realistic system responses but fundamentally lack dynamic engagement capabilities. To overcome this limitation, subsequent research introduced Reinforcement Learning (RL) [5, 6, 7, 8, 9] to equip honeypots with adaptive deception strategies. Recent advanced implementations [10, 11] even utilize real operating systems as backends. These systems aim to achieve maximum environmental fidelity alongside RL-driven dynamics.

However, these RL-driven approaches encounter a critical bottleneck regarding state formulation. RL models heavily rely on rigid state representations and fail to semantically comprehend the underlying intent behind multi-turn command sequences. This lack of intent awareness creates a significant gap in current honeypot designs. When a deceptive system cannot accurately interpret the actual objective of an ongoing intrusion, it inevitably generates irrelevant or illogical responses to attacker behaviors. Such behavioral inconsistencies alert sophisticated adversaries, which exposes the true nature of the honeypot and prematurely terminates the intelligence gathering process.

To bridge the critical gap in cross-turn intent awareness, leveraging LLMs represents a natural progression. Their inherent semantic reasoning capabilities offer a promising approach for extracting adversarial intent from sequential commands. However, directly deploying LLMs into honeypot environments introduces two fundamental challenges. First, raw command sequences generated during cyberattacks are inherently noisy and ambiguous. Relying on few-shot prompting to process such unstructured data often causes LLMs to hallucinate or funda-

mentally misinterpret the true adversarial intent. Second, even if the model successfully identifies the correct intent, a critical behavioral misalignment remains. The default conversational policies of LLMs are inherently helpful rather than deceptive and are therefore not optimized for honeypot scenarios. These general-purpose LLMs fundamentally lack the domain-specific engagement intuition required to sustain a credible interaction and strategically manipulate the attacker.

To systematically address the critical limitations regarding the lack of intent awareness and engagement intuition in current honeypot architectures, this research formulates two fundamental problems.

1. **Intent Awareness:** How can a deceptive system accurately extract and comprehend the true adversarial intent from highly noisy and multi-turn command sequences?
2. **Engagement Intuition:** How can we align LLMs toward a honeypot-specific intuition rather than relying on their suboptimal default conversational policies?

To address the identified research problems, we propose IntenSensePot, a novel intent-aware honeypot framework. This framework systematically transitions the operational role of an LLM from a generic responder to a specialized engagement strategist. We accomplish this transformation through a two-stage evolutionary training process following an initial phase of online data collection.

The first stage establishes robust intent awareness by internalizing the intricate mapping between multi-turn command sequences and the underlying adversarial intent. Subsequently, the second stage cultivates domain-specific engagement intuition by shifting the model from a response-only LLM to a proactive deceptive strategist. Ultimately, this dual-phase architecture enables a single LLM to accurately perceive attacker intent and seamlessly execute strategic deception to maximize the collection of high-value threat intelligence. This paper makes the following primary contributions:

- **End-to-End Framework Formulation:** Propose a comprehensive LLM honeypot frame-

work designed to orchestrate dynamic system configurations on a real backend operating system.

- **Intent-Awareness Chain-of-Thought Finetuning:** Embed adversarial intent extraction directly into the reasoning process of the LLM. This methodology bridges the semantic gap caused by noisy command sequences and ensures a reliable foundation for intent-aware engagement.
- **RFT-based Engagement Policy Alignment:** Systematically reshape the suboptimal default conversational policy of the LLM. This optimization enables dynamic strategy selection to deceive adversaries and draw them deeper into the simulated environment.
- **Empirical Findings:** Utilizing LLaMA-3.1-8B as our base model, our first phase improves the intent detection F1-score from 0.29 to 0.84 and elevates the intent-strategy alignment score from 0.54 to 0.78. This guarantees the coverage of most attack intentions within a single inference step. Building upon this precise perception, our second phase extends the maximum attacker engagement depth to 5.92 (compared to 1.32(348%) for QRASSH and 3.00(97%) for LLM-Honeypot) and increases the interaction diversity to 8.96 (compared to 3.66(144%) and 5.10(75%) for QRASSH and LLM-Honeypot, respectively). These results empirically prove that an LLM must align with a domain-specific policy to optimize overall deception efficiency.

The remainder of this paper is organized as follows: Section II reviews relevant background and related work. Section III establishes the formal problem formulation for intent-awareness and engagement intuition. Section IV presents our proposed intent-aware honeypot framework and its two-phase evolution process. Section V details the technical implementation, followed by an evaluation of experimental results in Section VI. Finally, Section VII concludes the paper and outlines potential directions for future research.

Chapter 2. Background & Related Work

2.1 Honeypot

Honeypots represent a fundamental technique within the active defense domain, primarily used to collect threat intelligence and observe adversary behaviors. Honeypots are generally categorized into three types: low, medium, and high interaction. Low-interaction honeypots, such as Honeyd[12], often simulate shallow services or protocols, including web server or Telnet. Consequently, they predominantly log probing activities and initial reconnaissance efforts instead of providing an attack surface for engagement.

Medium-interaction honeypots advance this concept by simulating a more interactive shell environment. The primary objective of these systems is to collect adversarial command sequences. Traditional implementations like Cowrie[1] and Kippo[13] are equipped with predefined scripts to simulate system responses alongside a restricted file system. While these systems are lightweight and easily deployable, their static nature limits continuous engagement. Recently, LLMs have been introduced to power medium-interaction honeypots[2, 3, 4]. These LLM-based approaches provide highly diverse system responses, significantly improving simulation variability compared to traditional script-based methods. However, LLMs inherently struggle to maintain complex, stateful system responses over prolonged sessions. Furthermore, because they lack an underlying execution environment, they remain incapable of processing essential adversarial behaviors such as payload downloading and binary execution.

To address the limitations of simulated environments, high-interaction honeypots are built upon real operating systems [10, 11]. These systems expose a fully functional environment to attackers, utilizing monitoring mechanisms to transparently record attack activities while yield-

ing system responses natively. Due to this high fidelity, attackers face difficulty in identifying the system as a honeypot. The trade-off for this realism is higher deployment and maintenance costs. Among these categories, we select high-interaction honeypots as the baseline for our research. Their fidelity provides the strongest foundation for adaptive engagement.

2.2 Reinforcement Learning in Honeypot

Reinforcement Learning (RL) is a technique utilized to improve an agent’s policy based on environmental feedback to achieve optimal rewards. In honeypot research, an engagement is typically formulated as a Markov Decision Process (MDP). Within this framework, attacker behavior is assumed to follow a specific probability distribution, allowing the defensive agent to learn the optimal transition probabilities during interactions to maximize its cumulative reward.

In previous studies, to enhance the dynamism of honeypot responses, RL was utilized to enable honeypots to select optimal response strategies during engagements. This approach attempts to entice attackers into revealing more attack samples, as seen in QRASSH[5], GASH[7], and [8, 9, 10, 11]. However, these RL model designs encounter a shared issue: neglecting contextual dependencies. They rely solely on a single, isolated command to determine the subsequent response, despite the fact that commands frequently possess varying parameter combinations. Without sufficient capability to accurately identify the state, the RL model is driven to exploit the highest-reward pattern rather than the contextual mapping between reward and action.

Consequently, by neglecting complex sequential semantics, traditional RL honeypots are restricted to rigid interactions that fail to sustain diverse adversary engagement. Overcoming this limitation requires a fundamental shift towards policies capable of interpreting complex attack intents and generating dynamic responses.

2.3 MITRE Framework

The MITRE ATT&CK[14] framework is an attack-centric knowledge base that maps observed adversary behaviors to Tactics, Techniques, and Procedures (TTPs). Tactics describe the high level goals of an attacker, while techniques and procedures detail the specific implementations, used to achieve those goals. Common adversary emulation frameworks, such as Sliver[15] and Atomic Red Team[16], align their capabilities with this model. Consequently, MITRE ATT&CK establishes a standardized taxonomy that facilitates clear communication between red and blue team operations. In our research, we leverage the MITRE ATT&CK framework to construct a temporally sequential severity matrix. This matrix models the natural progression of an intrusion, where tactical severity escalates along a temporal timeline. It serves two primary objectives: quantifying the engagement quality of the honeypot throughout the attack lifecycle, and mapping the adversary’s semantic intent to specific tactical stages. Consequently, the dynamic strategies generated by our honeypot are empirically grounded in observed adversarial progression, rather than relying on unverified heuristic outputs.

Tactics	Initial Access 10 techniques	Execution 10 techniques	Persistence 18 techniques	Privilege Escalation 12 techniques	Defense Evasion 25 techniques	Credential Access 15 techniques	Discovery 25 techniques	Lateral Movement 8 techniques	Collection 14 techniques	Command and Control 18 techniques	Exfiltration 8 techniques	Impact 15 techniques
Techniques	Content Injection Drive-by Compromise Exploit Public-Facing Application External Remote Services Hardware Additions Phishing (4) Supply Chain Compromise (3) Trusted Relationship Valid Accounts (9) Wi-Fi Networks	Command and Scripting Interpreter (2) Exploitation for Client Execution Input Injection Inter-Process Communication Native API Scheduled Task/Job (23) Shared Modules Software Deployment Tools System Services (1) User Execution (23)	Account Manipulation (2) Boot or Logon Initialization Scripts (1) Boot or Logon Autostart Execution (2) Compromise Host Software Binary Create Account (2) Create or Modify System Process (1) Event Triggered Execution (4) Exclusive Control External Remote Services Hijack Execution Flow (2) Process Injection (23) Modify Authentication Process (2) Power Settings Pre-OS Boot (2)	Abuse Elevation Control Mechanism (2) Account Manipulation (2) Boot or Logon Autostart Execution (2) Root or Logon Initialization Scripts (1) Create or Modify System Process (1) Escape to Host Exploitation for Privilege Escalation Hijack Execution Flow (2) Process Injection (23) Scheduled Task/Job (2) Valid Accounts (9)	Abuse Elevation Debugger Evasion Email Spoofing Execution Guardrails (2) Exploitation for Defense Evasion File and Directory Permissions Modification (1) Hide Artifacts (11) Hijack Execution Flow (2) Impair Defenses (7) Impersonation Indicator Removal (3) Masquerading (9) Modify Authentication Certificates Process (2)	Adversary-in-the-Middle (2) Brute Force (4) Credentials from Password Stores (2) Exploitation for Credential Access Forge Web Credentials (1) Input Capture (4) Multi-Factor Authentication Interception Multi-Factor Authentication Request Generation Network Sniffing OS Credential Dumping (3) Steal or Forge Authentication Certificates Valid Accounts (9)	Account Discovery (23) Application Window Discovery Browser Information Discovery Debugger Evasion (4) Device Driver Discovery File and Directory Discovery Log Enumeration Network Service Discovery Network Share Discovery Network Sniffing Password Policy Discovery Peripheral Device Discovery Permission Groups Discovery (2)	Exploitation of Remote Services Internal Spearphishing Lateral Tool Transfer Remote Service Session Hijack (1) Remote Services (2) Software Deployment Tools Taint Shared Content Use Alternate Authentication Material Data from Removable Media	Adversary-in-the-Middle (2) Archive Collected Data (3) Audio Capture Automated Collection Clipboard Data Data from Information Repositories Data from Local System Data from Network Shared Drive Data from Removable Media Data Staged (2) Email Collection (1) Input Capture (4) Screen Capture	Application Layer Protocol (3) Communication Through Removable Media Content Injection Data Encoding (2) Data Obfuscation (3) Clipboard Data Data from Information Repositories Data from Local System Data from Network Shared Drive Data from Removable Media Data Staged (2) Email Collection (1) Input Capture (4) Screen Capture	Automated Exfiltration Data Transfer Size Limits Exfiltration Over Alternative Protocol (2) Exfiltration Over C2 Channel Exfiltration Over Other Network Medium (1) Exfiltration Over Web Service (4) Multi-Stage Channels Non-Application Layer Protocol Non-Standard Port Protocol Tunneling Proxy (4)	Account Access Removal Data Destruction Data Manipulation (2) Data Encrypted for Impact Data Manipulation (2) Defacement (2) Email Bombing Endpoint Denial of Service (4) Financial Theft Firmware Corruption Inhibit System Recovery Network Denial of Service (2) Resource Hijacking (2) Service Stop System Shutdown/Reboot

Figure 1: MITRE ATT&CK Enterprise Matrix

To systematically model and evaluate defensive strategies, this research leverages the MITRE Engage framework[17]. MITRE Engage is a defender-centric framework that provides a standardized taxonomy for deception and denial operations, facilitating the critical transition from passive detection to active defense. By offering an empirical mapping between adversary behaviors and defensive tactics, it enables defenders to strategically select appropriate engage-

ment activities based on their specific operational objectives.

At its core, the framework structures engagement operations around three primary strategic goals: **Expose**, **Affect**, and **Elicit**. Specifically, **Expose** operations aim to trigger observable adversary behaviors that reveal their presence and capabilities to the defender. **Affect** operations focus on altering the engagement environment to negatively impact the adversary's progression, such as wasting their resources or degrading their confidence. Furthermore, **Elicit** operations are designed to strategically manipulate the attacker's decision-making process to extract specific TTPs or actionable threat intelligence.

In alignment with this taxonomy, the system design presented in **Chapter 4** of this thesis utilizes specific engagement activities, including *Lures*, *Artifact Diversity*, *Software Manipulation*, and *Introduced Vulnerabilities*. These selected activities primarily fall under the **Affect** and **Elicit** goals, as our underlying objective is to dynamically disrupt the attacker's operational flow while maximizing the extraction of complex threat intelligence during the engagement.

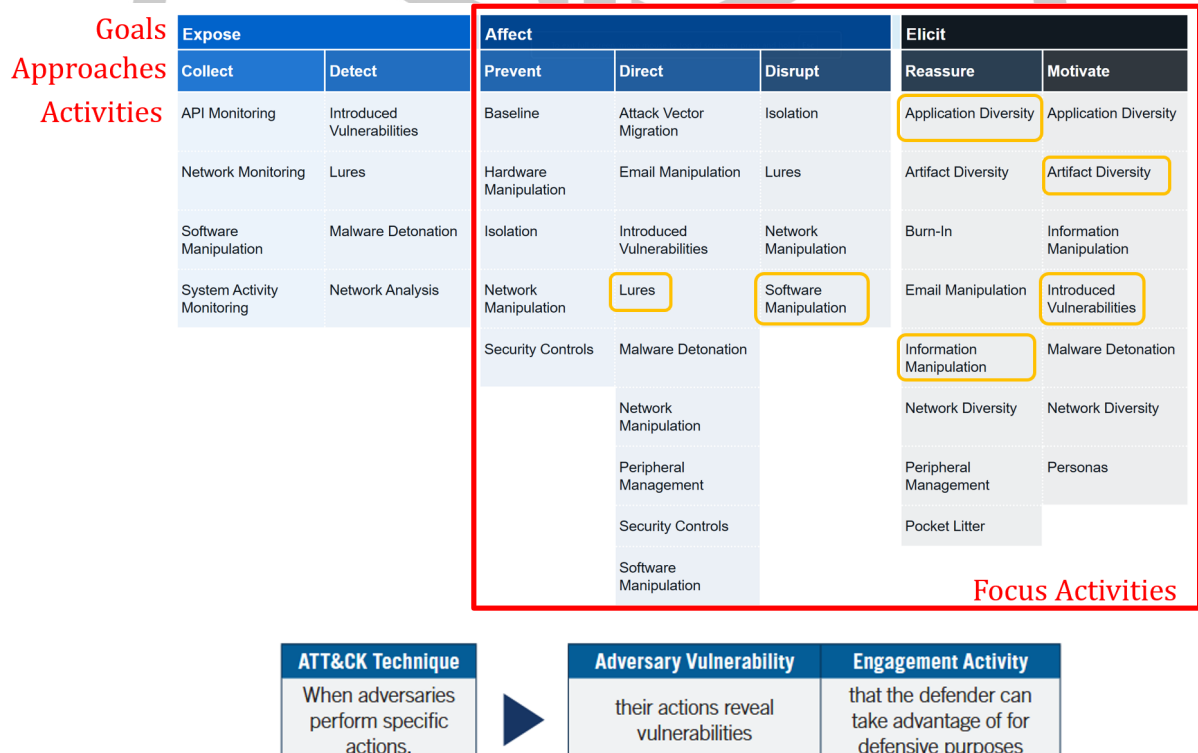


Figure 2: MITRE Engage Matrix

2.4 Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) is an efficient reinforcement fine-tuning method designed for LLMs. The core concept of this approach involves providing a single prompt to the model, which then generates multiple candidate responses. The algorithm calculates the relative advantage of these responses based on the mean and standard deviation of the group’s rewards. By evaluating advantages within this group, GRPO updates the policy weights without requiring a separate critic network. This architectural choice significantly reduces memory costs and mitigates reward distribution shifts, thereby enhancing convergence stability during the training process.

Standard LLMs are typically aligned toward safe and helpful conversational behavior through reinforcement learning from human feedback [18]. This alignment process, while improving helpfulness and safety in general-purpose settings, instills a systematic refusal tendency toward adversarial or security-sensitive inputs [19], making them generally unsuitable for direct application in honeypot engagements where the system must continuously process and strategically respond to adversarial commands. Qi et al. [20] demonstrate that even limited fine-tuning can substantially override such safety alignment. While their work highlights this as a security risk, we deliberately exploit this property to redirect the model’s policy from its default helpful behavior toward a deceptive engagement strategy. GRPO is particularly suited for this redirection, as its critic-free, group-relative advantage estimation enables stable policy updates with lower memory overhead, allowing us to efficiently reshape the LLM’s behavior to meet the specific requirements of the active defense domain.

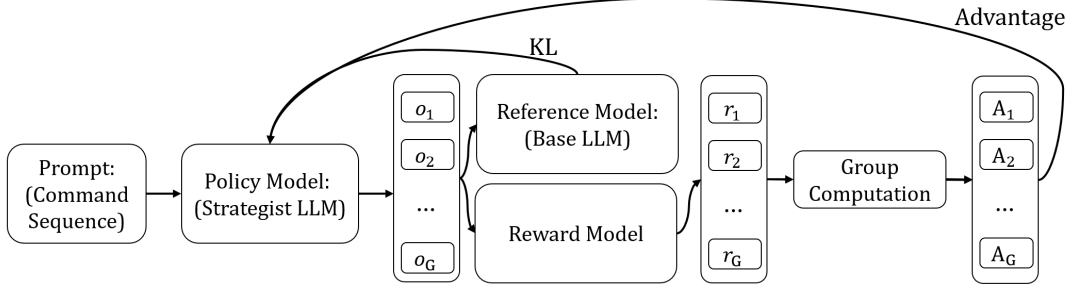


Figure 3: Reinforcement Finetuning - GRPO

2.5 Related Work

Our research focuses on leveraging fine-tuning to improve intent-awareness and optimal strategy selection within a single LLM. Existing adaptive honeypot research can be organized into three architectural categories based on how the LLM and RL components interact, against which we position our proposed Unified RL-LLM Honeypot as a fourth category.

The first category consists of **Prompt-based LLM Honeypots** that rely solely on LLMs without any reinforcement learning component. HoneyGPT [2], for instance, achieves adaptivity through structured prompt engineering with chain-of-thought reasoning and memory pruning. However, its behavior is entirely governed by the prompt design, without any reward signal tied to honeypot-specific goals such as engagement depth or technique diversity, motivating the integration of RL with LLM-based honeypots.

In the **Policy-as-Strategist** approach [7, 8, 9], the LLM acts primarily as a backend environment simulator. An RL agent is responsible for outputting high-level strategic decisions, often using frameworks like MITRE Engage or ABSI to guide the response style of the LLM. However, the RL model typically makes decisions based on a single command state. This separation decouples the strategic decision-making from the semantic understanding of the LLM, ultimately resulting in a perception-decision gap.

Alternatively, the **Policy-as-Executor** approach [1, 10, 11] treats the policy model differently. While specific systems like QRASSH [5] utilize a single model to handle direct

command-to-response mapping, most systems in this category use the LLM merely as a detection method. The LLM maps a single command into TTPs, which then serve as the state input for the RL agent. Subsequently, the RL agent executes specific and limited actions on a dedicated backend system. This architecture is heavily restricted by a system-dependent and narrow operational action space. Due to this inconsistent model collaboration in decoupled architectures, the RL model often converges toward relatively safe and rigid outcomes. Consequently, this structural limitation constrains and ultimately wastes the inherent generative and analytical capabilities of the LLM.

Table 1 summarizes the comparison across six dimensions: the role of the policy model, RL design (state/action/reward), detection method, modified target, real-system backing, and intent aware.

Table 1: Comparison of Adaptive Honeypot systems

Category	Paper	RL Setup				Modified Target	Detection Method	Real System	Intent Aware
		State	Action	Reward					
				Engagement Depth	Technique Diversity				
LLM Honeypot	HoneyGPT[2]	No RL (Prompt-based)				Response	LLM	-	✓
Policy-as-Strategist	Chris Huang[9]	Technique	MITRE Engage	✓	-	Response	LLM	-	-
Policy-as-Executor	QRASSH[5]	Command	ABSI	Command Type		Response	DNN	-	-
	Chun Yu[10]	Technique	MITRE Engage	✓	-	Response	LLM	✓	-
	Rasul[11]	Tactic	MITRE Engage	✓	-	Response	LLM	✓	-
Unified RL-LLM	Ours	Command sequence	MITRE Engage	✓	✓	System configuration	LLM	✓	✓

Our proposed solution is categorized as a Unified RL-LLM Honeypot. We employ a single LLM to act as the core strategist, which bridges the perception-decision gap typically found in dual-model architectures. By consistently ingesting command sequences, the single LLM can output intent-awareness strategies. These strategies are then executed using a wider

range of applicable tools on a real system backend. Finally, our system utilizes a reward design composed from the attacker’s perspective. This mechanism encourages the strategist LLM to select optimal engagement strategies based on inferred intentions, thereby capturing more diverse and deeper attack patterns.



Chapter 3. Problem formulation

This chapter establishes the formal problem formulation for our study, modeling the adversarial engagement as a dynamic, sequential decision-making process. Section 3.1 introduces the fundamental mathematical notations used throughout the paper, while Section 3.2 formulates our two primary research problems: accurately detecting adversarial intent from noisy command contexts and optimizing the LLM’s deceptive strategy.

3.1 Notation Table

To formally model the dynamic engagement between the adversary and our deceptive environment, we establish a set of foundational notations categorized by their operational roles in Table 2. The fundamental unit of this interaction is defined within the common category as a trajectory $\mathcal{T}$, which encapsulates the entire contextual sequence of states, actions, and rewards within a single session. Throughout this trajectory, the interaction is driven by a total of N_c commands, where each i -th step corresponds to a specific input command C_i from the attacker. To maintain the continuous interactive state, these sequential inputs are aggregated and tracked within a command buffer C_B .

From the perspective of the honeypot system, managing the adversary requires bounded contextual awareness and strategic decision-making. The system operates within a defined command window size N , which must strictly adhere to the underlying LLM’s maximum context window limit W . By tracking C_i^S , which isolates the commands that practically affect the sandbox environment, the system attempts to comprehend the ongoing attack progression. Based on the recent sequence of inputs, the LLM-driven module detects $\hat{z}_i$, representing the detected semantic intent and tactical objectives. This understanding dictates the subsequent

strategy, allowing the policy to select an optimal deceptive action $a_i \in A$ and generate the corresponding system response R_i . The defender’s toolkit specification S_D defines the set of available deception tools, providing the formal interface through which selected strategies are translated into executable system manipulations.

In contrast to the system’s internal estimations, the attacker category defines the absolute ground truth of the adversarial operation. At any given step i , the actual semantic intent and tactical objective harbored by the attacker are formally denoted as z_i . This explicit formulation is mathematically critical, as the discrepancy between the detected intent $\hat{z}_i$ and the adversary’s true intent z_i forms the fundamental basis for defining the intent detection objective of our unified LLM policy. The attacker’s toolkit specification S_A defines the set of technique-grounded tools available to the attacker, providing the formal interface through which attack intentions are translated into concrete system commands.

Finally, to iteratively refine the LLM’s deceptive capabilities, we define a set of performance notations centered around reinforcement fine-tuning. The overarching objective of the policy is to maximize a composite environmental reward, denoted as R_{env} . This reward structure is driven by two critical security metrics: the engagement depth S_{depth} , quantified by the cumulative tactic ID leveraged by the adversary, and the strategy diversity S_{div} , measuring the number of unique techniques elicited during the session. These two components are integrated using designated weighting coefficients w_j , mathematically formulating the ultimate incentive structure that guides the honeypot agent.

Table 2: Notations

Category	Notation	Description
Common	$\mathcal{T}$	A trajectory representing the full engagement context of a single session, typically defined as a sequence of states, actions, and rewards.
	N_c	Total number of commands in one trajectory, where index $i \in \{1, 2, \dots, N_c\}$.
	C_i	The i -th input command from the attacker in one trajectory.
	C_B	Commands currently stored in the command buffer.
Honeypot system	N	Command window size.
	W	Context window size of the LLM.
	$\hat{z}_i$	Detected semantic intent and tactics based on the sequence $C_{i-N}, \dots, C_i$.
	$a_i \in A$	The action decided by the LLM for the sequence $C_{i-N}, \dots, C_i$.
	C_i^S	The sandbox configuration script translated by the MCP Toolkit at i -th step.
	R_i	The i -th system response from the honeypot for the sequence $C_{i-N}, \dots, C_i$.
	S_D	Defender’s toolkit specification, including deception tool implementation.
Attacker	z_i	Attacker semantic intent and tactic at the i -th input command.
	S_A	Attacker’s toolkit specification, including command implementation of technique.
Performance	w_j	The weight assigned to the reward components, where index $j \in \{1, 2\}$.
	$\mathcal{S}_{\text{depth}}$	Cumulative weighted-tactic ID used by the adversary in one trajectory
	$\mathcal{M}_{\text{depth}}$	A maximum threshold for the cumulative tactic severity.
	$\mathcal{S}_{\text{div}}$	Number of unique techniques used in one trajectory.
	$\mathcal{M}_{\text{div}}$	Total number of commands execution in one trajectory
	R_{env}	Composite reward: $w_1 \cdot \text{Norm}(\mathcal{S}_{\text{depth}}, \mathcal{M}_{\text{depth}}) + w_2 \cdot \text{Norm}(\mathcal{S}_{\text{div}}, \mathcal{M}_{\text{div}})$.

3.2 Problem Statement

Building upon the established notations, we formally define the operational framework of our honeypot system. The interaction between the adversary and the LLM-driven environment is framed around two sequential, interdependent problems: accurately detecting the underlying adversarial intent from noisy inputs and dynamically selecting an optimal deceptive strategy. Although formulated separately for clarity, both problems are addressed within a single inference pass of the Strategist LLM, where the detected intent serves as an intermediate Chain-of-Thought that conditions the subsequent strategy selection.

Problem 1. Accurately Detect Intent from Command Sequences

The first objective of the system is to accurately map a sequence of raw adversarial commands to a high-level semantic intent and tactical classification.

- **Input:**
 - $(C_{i-N}, \dots, C_i)$: A historical command sequence within the defined window
- **Output:**
 - $\hat{z}_i$: The detected semantic intent and associated MITRE ATT&CK tactic
- **Objective:**
 - Maximize the intent detection rate by minimizing the divergence between the detected intent $\hat{z}_i$ and the attacker’s ground truth intent z_i
- **Constraint:**
 - $\text{length}(C_{i-N}, \dots, C_i) \leq W$: The total token length of the command sequence must strictly adhere to the LLM’s context window limit

Research Challenge: Mapping raw commands to high-level intents is fundamentally hindered by semantic ambiguity and operational noise, such as frequent syntactic errors and typos. Cur-

rently, the command-to-tactic mapping is non-bijective: a single command’s purpose depends heavily on the surrounding context and attack stage, while multiple commands often chain together to form a compound tactic. Thus, we frame intent detection as a contextual sequence-to-sequence understanding problem rather than naive discrete classification.

Problem 2. Optimized Strategy Selection

Conditioned on the accurately detected intention, the honeypot must subsequently determine the most effective response strategy to securely engage and manipulate the adversary.

- **Input:**

- $\hat{z}_i$: The detected semantic intent and tactic.
- $(C_{i-N}, \dots, C_i)$: The historical command sequence.

- **Output:**

- a_i : An optimal deceptive strategy aimed at deceiving the attacker.

- **Objective:**

- Maximize the composite environmental reward R_{env} , formulated as a convex combination of normalized tactic severity and technique diversity:

$$R_{\text{env}} = w_1 \cdot \text{Norm}(\mathcal{S}_{\text{depth}}, \mathcal{M}_{\text{depth}}) + w_2 \cdot \text{Norm}(\mathcal{S}_{\text{div}}, \mathcal{M}_{\text{div}}) \quad (3.1)$$

where the normalization function is defined as $\text{Norm}(x, \mathcal{M}) = \min(1, \frac{x}{\mathcal{M}})$, subject to the weight constraint $w_1 + w_2 = 1$ with $w_1, w_2 \in [0, 1]$.

- **Constraint:**

- $\text{len}(C_{i-N}, \dots, C_i) + \text{len}(\hat{z}_i) \leq W$: The combined token length of the command sequence and detected intent must remain within the LLM’s context window capacity.

Research Challenge: The core challenge in strategy selection is effectively mapping the detected adversarial intent to an optimal defensive action. Pre-trained LLMs are intrinsically aligned with a “helpful and honest” default policy. However, this alignment is fundamentally sub-optimal for honeypot operations. If the system relies on this default policy, the wrong strategy may be chosen, leading to premature exposure of the decoy and, consequently, low engagement efficiency. Therefore, the critical challenge lies in shifting the LLM’s paradigm toward a purely deceptive policy, enabling it to properly engage with the adversary and maximize the generation of actionable threat intelligence.



Chapter 4. Solution Approach

This chapter presents *IntenSensePot*, our proposed framework for transforming a general-purpose LLM into a domain-specialized honeypot engagement strategist. To address the two research problems defined in Chapter 3, the framework is structured around two sequential phases: online data collection and offline optimization. Figure 4 illustrates the overall architecture and training pipeline.

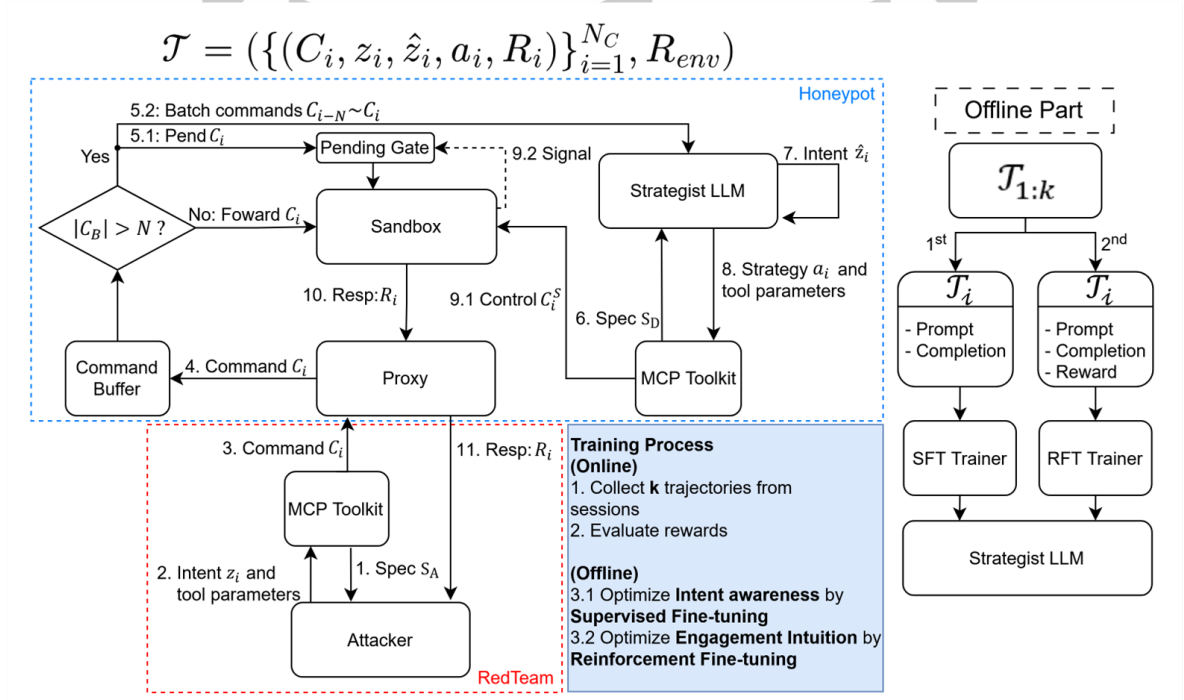


Figure 4: Solution Framework

4.1 Solution Overview

To simultaneously establish accurate *Intent-awareness* and cultivate *Engagement Intuition*, our framework operates across two distinct phases.

Phase 1: Online Data Collection. We construct an adversarial engagement sandbox to capture

realistic attack behaviors and defensive interactions. In this phase, a Red Team Agent acts as the attacker. The Red Team Agent is an LLM-driven autonomous agent whose implementation details are described in Chapter 5. It dynamically issues command sequences, while our defending Strategist LLM intercepts these commands, infers the underlying adversarial intent, and deploys deceptive strategies via a defined toolkit. As illustrated in Figure 4, the complete interaction sequence of commands, intents, selected strategies, and system responses is recorded as a trajectory, and a terminal environmental reward is assigned to evaluate the overall engagement quality.

Phase 2: Offline Optimization. Using the collected interaction data, the Strategist LLM undergoes a two-stage offline optimization process. The first stage employs Supervised Fine-Tuning to build reliable intent-awareness, ensuring the model can accurately map raw command sequences to tactical objectives. The second stage applies Reinforcement Fine-Tuning to optimize the model’s decision-making policy, utilizing the environmental reward signals to foster long-term engagement intuition. Ultimately, the Strategist LLM evolves into a unified agent capable of both accurate threat perception and optimal deceptive engagement. The details of each component are elaborated in the following sections.

4.2 Supervised Fine-Tuning for Intent-Aware Chain-of-Thought

Accurately inferring adversarial intent from raw command sequences is a prerequisite for effective strategy selection. Without this capability, the Strategist LLM risks hallucinating or misinterpreting the attacker’s objectives, propagating errors into all subsequent strategic decisions. Beyond correctness, the inferred intent serves a structural role as an explicit Chain-of-Thought that conditions the model’s reasoning before a deceptive strategy is selected. As illustrated in Figure 5, given an observed command sequence, the Strategist LLM is expected to first produce a natural language description of the adversary’s probable objective, then identify the corresponding MITRE ATT&CK tactics, before arriving at a strategy decision. Optimizing this reasoning chain is therefore not an isolated objective but a foundational requirement for

the entire engagement pipeline.

To construct high-quality supervision signals for this stage, we employ a knowledge distillation approach. During the online data collection phase, the Red Team Agent is designed to explicitly reason over environment responses before selecting tools and issuing commands, producing a traceable decision process that reflects its true tactical intent. The full interaction context of each trajectory, including the agent’s reasoning, tool selections, and issued commands, is recorded in its entirety.

A commercial LLM is then employed as an Oracle Annotator. A valid concern in such pipelines is the potential for circular reasoning, given that both the Red Team Agent and the Oracle Annotator are LLM-driven. We mitigate this by ensuring the ground truth tactic at each step is strictly anchored by the Red Team Agent’s explicit tool selection from $\mathcal{S}_A$, where each tool is deterministically bound to a specific MITRE ATT&CK technique. Consequently, the Oracle Annotator’s role is not to infer intent from ambiguous raw behaviors, but rather to utilize the complete trajectory context to translate these already grounded tool-level decisions into structured natural language descriptions and tactic labels.

Although both the Red Team Agent and the Oracle Annotator are LLM-driven, the annotation process does not introduce circular reasoning. The ground truth intent at each step is anchored by the Red Team Agent’s explicit tool selection from $\mathcal{S}_A$, where each tool is bound to a specific MITRE ATT&CK technique. The Oracle Annotator’s role is therefore not to infer intent from ambiguous behavior, but to translate the already grounded tool-level decisions into structured natural language descriptions and tactic labels suitable for supervised training.

From these high-quality annotations, we construct supervised training pairs where the input consists solely of the observed command sequence without ground-truth intent, and the target output is the precisely labeled intent and tactic sequence. This asymmetry between the full trajectory and grounded tool decisions available at annotation time and the raw commands only available at inference time. It forces the Strategist LLM to internalize the complex mapping from surface-level commands to high-level adversarial reasoning, rather than relying on

contextual cues that would be unavailable during deployment.

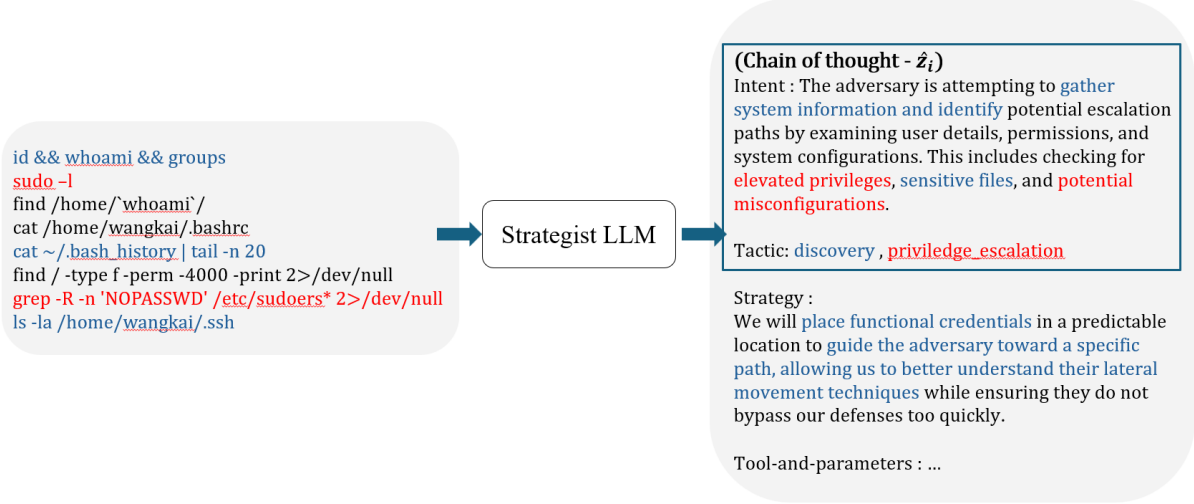


Figure 5: Construction of Chain of Thought (CoT)

4.3 Action Space

Prior RL-based honeypot systems treat the policy model as an executor, binding it to a fixed set of low-level system commands as its action space [10, 11]. This design creates two compounding problems. The action space is inherently system-dependent, and the policy cannot generalize across environments without retraining. To overcome these limitations, we decouple high-level engagement strategy from low-level execution by leveraging the Model Context Protocol (MCP) toolkit [21]. Under this design, the Strategist LLM selects actions at the level of abstract engagement intent, while the MCP Toolkit translates each selected strategy into parameterized tool invocations that operate on the underlying sandbox. When the system is migrated to a different environment, only the toolkit’s parameterized tools require updating, leaving the learned policy intact.

For reinforcement fine-tuning to converge reliably, the action space must be explicit and bounded. An unconstrained LLM can generate arbitrary natural language responses, many of which correspond to no executable action, causing the policy gradient to chase unrealizable strategies. To address this, we define five discrete engagement primitives as the primary action space, each grounded in the MITRE Engage framework [17]. As shown in Table 3, each primitive maps to a distinct subset of MITRE Engage activities, ensuring that every selectable

action has a concrete operational interpretation.

- **Partial Fail** presents controlled technical obstacles within the attack surface, simulating realistic system resistance through software manipulation.
- **Artifact Trail** deploys files, configurations, or activity traces that simulate the presence of legitimate human activity, drawing the attacker toward fabricated evidence.
- **Inert Admin Service** exposes services that appear to be legitimate administrative or operational endpoints, exploiting the attacker’s tendency to target high-value interfaces.
- **Credential Artifact** plants functional-looking credentials to redirect attacker focus and elicit credential-oriented attack behaviors.
- **Access Control Trap** introduces deliberate operational misconfigurations or vulnerabilities to entice exploitation attempts.

Table 3: Action primitives and their corresponding MITRE Engage activities.

Action List	Inspired by MITRE Engage Activities							
	Software Manipulation	Lures	Artifact Diversity	Pocket Litter	Application Diversity	Information Manipulation	Attack Vector Migration	Introduce Vulnerabilities
Partial Fail	✓							
Artifact Trail		✓	✓	✓				
Inert Admin Service		✓			✓			
Credential Artifact		✓				✓	✓	
Access Control Trap								✓

As illustrated in Figure 6, at each decision step the Strategist LLM receives the detected intent $\hat{z}_i$, the engagement activity descriptions, and the MCP toolkit corpus as context, and outputs a selected strategy along with the corresponding tool and execution parameters. Constraining the output to these five primitives prevents the model from generating operationally

impossible strategies during fine-tuning, improving both convergence stability and deployment reliability.

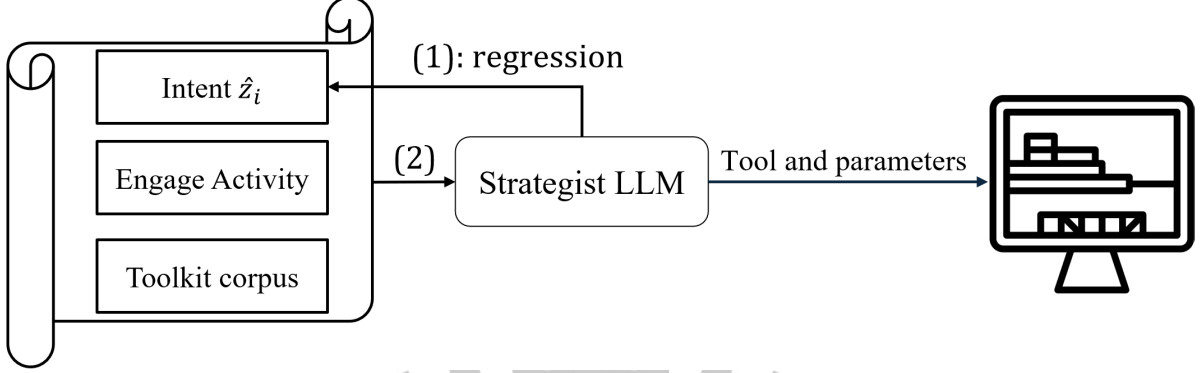


Figure 6: Action inference pipeline

4.3.1 Reward Design

Standard reward designs over MITRE ATT&CK often treat the tactic identifier as an ordinal severity indicator, scaling reward weights linearly with the tactic’s position in the matrix. This practice conflicts with the framework’s own definition: MITRE explicitly characterizes tactics as a set of categorical objectives whose ordering and completeness vary across intrusions, depending on what the adversary aims to achieve at each moment of the operation [14]. Beyond this definitional mismatch, linear weighting induces policy degeneration in reinforcement learning agents, which learn to prioritize high-ID tactics at the expense of the engagement diversity that constitutes a honeypot’s primary intelligence value. This concern is reflected in how prior ATT&CK-aligned reward designs have deliberately avoided linear identifiers; Q-Cowrie [22], for example, maps observed commands to ATT&CK lifecycle states but assigns reward through a severity tier scheme decoupled from tactic position.

We instead reorganize ATT&CK from a threat intelligence perspective rather than its temporal position. The seven tactics relevant to our honeypot are grouped by the adversary’s strategic intent at each stage, summarized in Table 4: *Data Collection* (Discovery, Collection, Credential Access), in which the adversary gathers information without altering its own operational state, through commands such as `whoami`, `cat .bash_history`, or reading

/etc/shadow; *Expansion* (Privilege Escalation, Lateral Movement), in which the adversary’s state transitions to a broader scope of control via actions like `sudo -l` or initiating SSH connections to internal hosts; and *Goals Achieved* (Exfiltration, Impact), in which the adversary realizes the intended outcome through irreversible operations such as transmitting data via `curl` or `scp`, or executing destructive commands like `rm -rf`.

Table 4: Attack Stages

Data collection		Expansion			Goals achieved	
1	2	3	4	5	6	7
Discovery	Collection	Credential Access	Privilege Escalation	Lateral Movement	Exfiltration	Impact

We instantiate this design as a normalized composite of two complementary signals, engagement depth and behavioral diversity:

$$R_{\text{env}} = w_1 \cdot \text{Norm}(\mathcal{S}_{\text{depth}}, \mathcal{M}_{\text{depth}}) + w_2 \cdot \text{Norm}(\mathcal{S}_{\text{div}}, \mathcal{M}_{\text{div}}), \quad (4.1)$$

where $\text{Norm}(x, \mathcal{M}) = \min(1, x/\mathcal{M})$ and $w_1 + w_2 = 1$. The depth score $\mathcal{S}_{\text{depth}} = \sum_{i=1}^{N_c} \mathcal{W}(\text{ID}_i)$ accumulates a non-linear weight $\mathcal{W}(\cdot)$ over the executed tactic sequence, with weights $\{0.1, 0.5, 2.0, 5.0, 8.0, 15.0, 20.0\}$ assigned to tactics 1 through 7 such that inter-stage gaps exceed intra-stage gaps. The diversity score $\mathcal{S}_{\text{div}} = |\{\Phi(C_i) \mid i \in [1, N_c]\}|$ uses set cardinality to count distinct ATT&CK techniques invoked, so that repeated invocation of the same technique increases N_c but leaves $\mathcal{S}_{\text{div}}$ unchanged. Normalizing both components to a common $[0, 1]$ scale serves two purposes: it converts w_1 and w_2 into interpretable trade-off ratios between depth-oriented and diversity-oriented engagement, and it bounds the contribution of each dimension so that no single signal can dominate cumulative return through unbounded accumulation. The saturation threshold $\mathcal{M}_{\text{depth}} = 35.0$ is selected so that the depth signal saturates only once the adversary has completed most of the kill chain, after which diversity becomes the primary remaining source of reward; the threshold $\mathcal{M}_{\text{div}} = 32$ corresponds to the maximum number of commands the system permits per episode and is therefore intrinsic to the environment.

Together, these two saturation mechanisms force the agent to optimize for progressively severe and behaviorally diverse engagements rather than exploiting repetitive high-tier loops.

4.4 Reinforcement Fine-Tuning for Engagement Intuition

Although the SFT stage reshapes the model’s behavior toward intent-aware responses, the resulting policy is not guaranteed to be optimal for honeypot engagement. The model may correctly identify attacker intent yet still default to strategy selections that fail to sustain deep or diverse adversarial behavior. The second optimization stage therefore applies reinforcement fine-tuning to further shift the policy toward maximizing the environmental reward.

Applying reinforcement learning directly in the online engagement environment proved impractical. The environmental reward R_{env} is a terminal signal assigned only upon session completion, providing no intermediate feedback that can be attributed to individual strategy selections within a multi-turn trajectory. This sparse reward structure, combined with the credit assignment difficulty inherent in complex sequential responses, prevented stable policy convergence during online training. To avoid the prohibitive cost of collecting additional online trajectories, we transfer the optimization entirely to an offline setting.

As shown in Figure 7, the offline pipeline proceeds in two steps. In the first step, a reward model is trained on the collected trajectory store under the *outcome-supervised* paradigm [23], in which the supervision signal is the trajectory-level terminal reward rather than per-step process annotations. Each trajectory carries a pre-labeled terminal reward R_{env} , which allows the reward model to learn the statistical relationship between observed command sequences, selected strategies, and engagement outcomes. This outcome-supervised formulation is well suited to our setting because the honeypot environment only emits a terminal reward upon session completion, making per-step process labels unavailable by construction. The trained reward model serves as a proxy environment, predicting the expected reward for any given (command sequence, strategy) pair without requiring live sandbox interaction. In the second step, the Strategist LLM is optimized via GRPO against this proxy. For each prompt drawn

from the offline trajectories, the model generates a group of N candidate strategy responses. The reward model scores each candidate, and the relative advantage within the group provides the gradient signal that nudges the policy toward higher-reward strategy selections. This design maximizes the reusability of collected trajectory data while fully decoupling policy optimization from the operational environment.

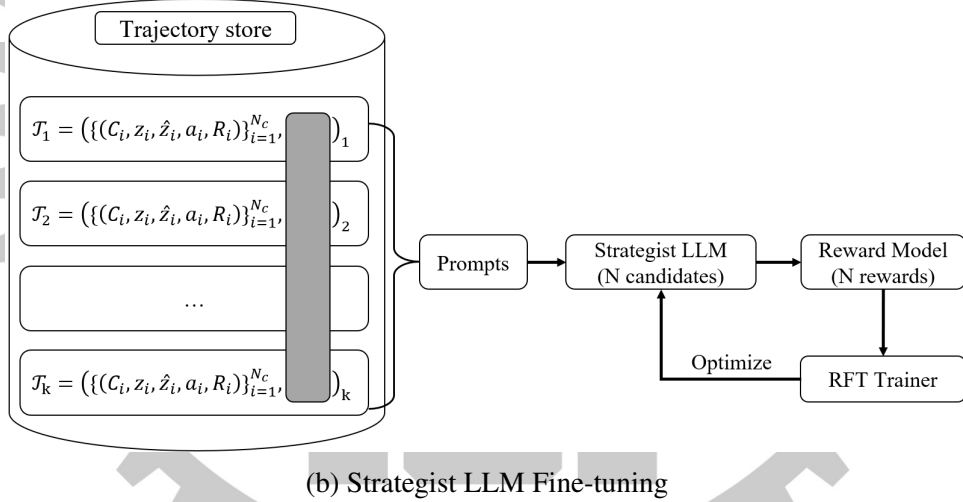
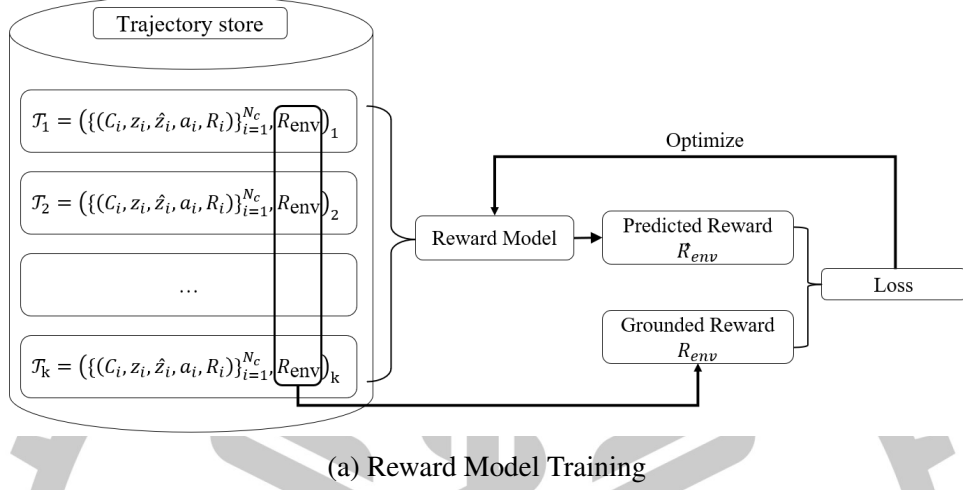


Figure 7: Reinforcement Fine-tuning Pipeline

Chapter 5. Implementation

5.1 System Deployment and Honeypot Architecture

The honeypot architecture, illustrated in Figure 8, comprises three components, namely the Strategist LLM, the MCP Toolkit, and the Simulated IT System. A MITM proxy intercepts every command issued by the attacker, and the strategist LLM is triggered for inference once 8 commands have been accumulated. Rather than rewriting the response returned to the attacker, the selected tool modifies the underlying environment by generating files, configurations, or artifacts consistent with the inferred intent, which the attacker then discovers naturally through subsequent reconnaissance. This indirect realization avoids hallucinated command outputs that would otherwise compromise the consistency of the environment.

In our implementation, the Strategist LLM does not directly synthesize shell outputs. Instead, it modifies the backend environment through MCP tools, while all outputs observed by the attacker are generated by the real sandbox. The engagement loop proceeds as follows. First, the MITM proxy records each attacker command and the corresponding sandbox response. Second, once the command buffer reaches eight commands, the Strategist LLM receives the buffered command history and predicts the attacker's intent and the corresponding deception strategy. Third, the selected MCP tool modifies the sandbox state by creating artifacts, changing permissions, launching inert services, or planting credentials. The attacker then observes these changes only through subsequent commands.

The Defender MCP Toolkit defines the action space available to the strategist and follows the five strategic dimensions of MITRE Engage, including Partial Failed, Artificial Trails, Inert Admin Surface, Credential Artifact, and Access Control Trap. Each strategy is mapped

to exactly one tool with a matching name so that the LLM concentrates on strategy selection, while diversity in deployment is expressed through the tool parameters populated based on the inferred context. Table 5 summarizes the mapping. For instance, when the attacker repeatedly relies on `ls` for system discovery, the strategist may invoke `mcp_partial_fail` with `target_tool=ls` and `failure_mode=permission_denied` to force the attacker toward alternative tools and elicit additional behavioral signals.

Table 5: MCP Tool Specification for Honeygot Strategy Primitives

Tool ID	Strategy Primitive	Tool Parameters
<code>mcp_partial_fail</code>	PARTIAL_FAIL: Present technical obstacles within the attack surface	<code>target_tool</code> : <command> (dynamically determined by LLM based on attacker behavior) <code>failure_mode</code> : <code>permission_denied</code> <code>missing_library_error</code>
<code>mcp_artifact_trail</code>	ARTIFACT_TRAIL: Create files, configurations, or activity traces that simulate realistic human traces	<code>persona_type</code> : <code>db_admin_maintenance</code> <code>devops_debugging</code> <code>legacy_sysadmin</code> <code>clueless_user</code>
<code>mcp_inert_admin_surface</code>	INERT_ADMIN_SURFACE: Launch service that seems to be a legitimate administrative or operational endpoint	<code>surface_type</code> : <code>mysql_db</code> <code>internal_http_portal</code> <code>legacy_telnet</code> <code>custom_rpc</code>
<code>mcp_credential_artifact</code>	CREDENTIAL_ARTIFACT: Place functional credentials	<code>template_id</code> : <code>sysadmin_setup_script</code> <code>developer_todo_note</code>
<code>mcp_access_control_trap</code>	ACCESS_CONTROL_TRAP: Make operational mistake or vulnerability	<code>target_binary</code> : <command> (dynamically determined by LLM based on attacker behavior)

The simulated IT system is built with container technology and includes a web server, a cache service, a database, and an Ubuntu shell preinstalled with commonly used utilities. Network paths between services are visible by default, providing the attacker with sufficient surface to explore from the start of the session through commands such as `curl` against the

web server or direct CLI connections to the cache service and the database.

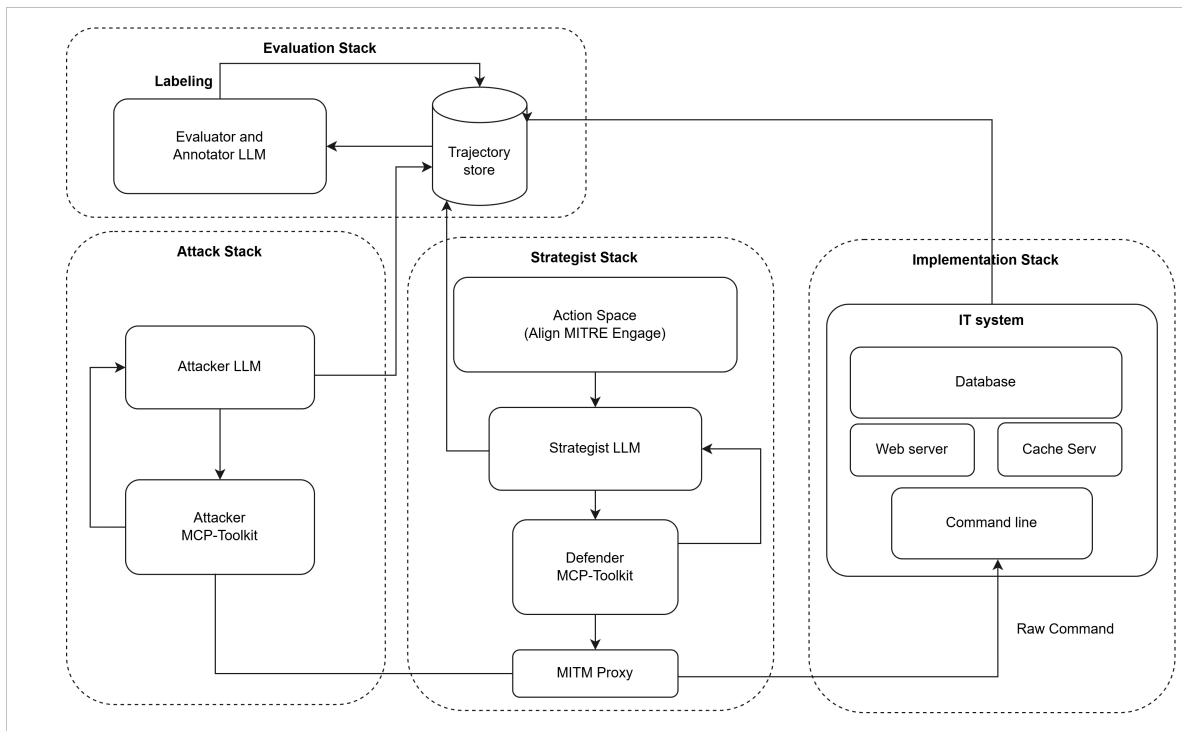


Figure 8: Honeypot architecture

5.2 Red Team Agent and Trajectory Collection

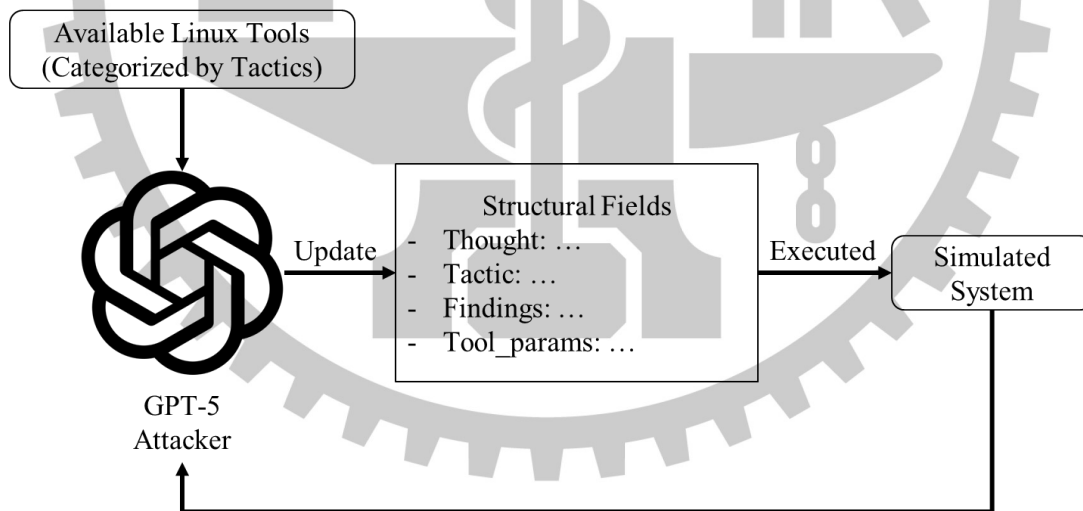


Figure 9: Red Team Agent Structural Output

The Red Team Agent is designed to progressively compromise the simulated environment and maximize the actionable attack behavior exposed during an engagement. Rather than following a fixed script, the agent reasons over its accumulated findings to advance toward high-value objectives such as exfiltration, and impact. This goal-oriented design enables adaptive

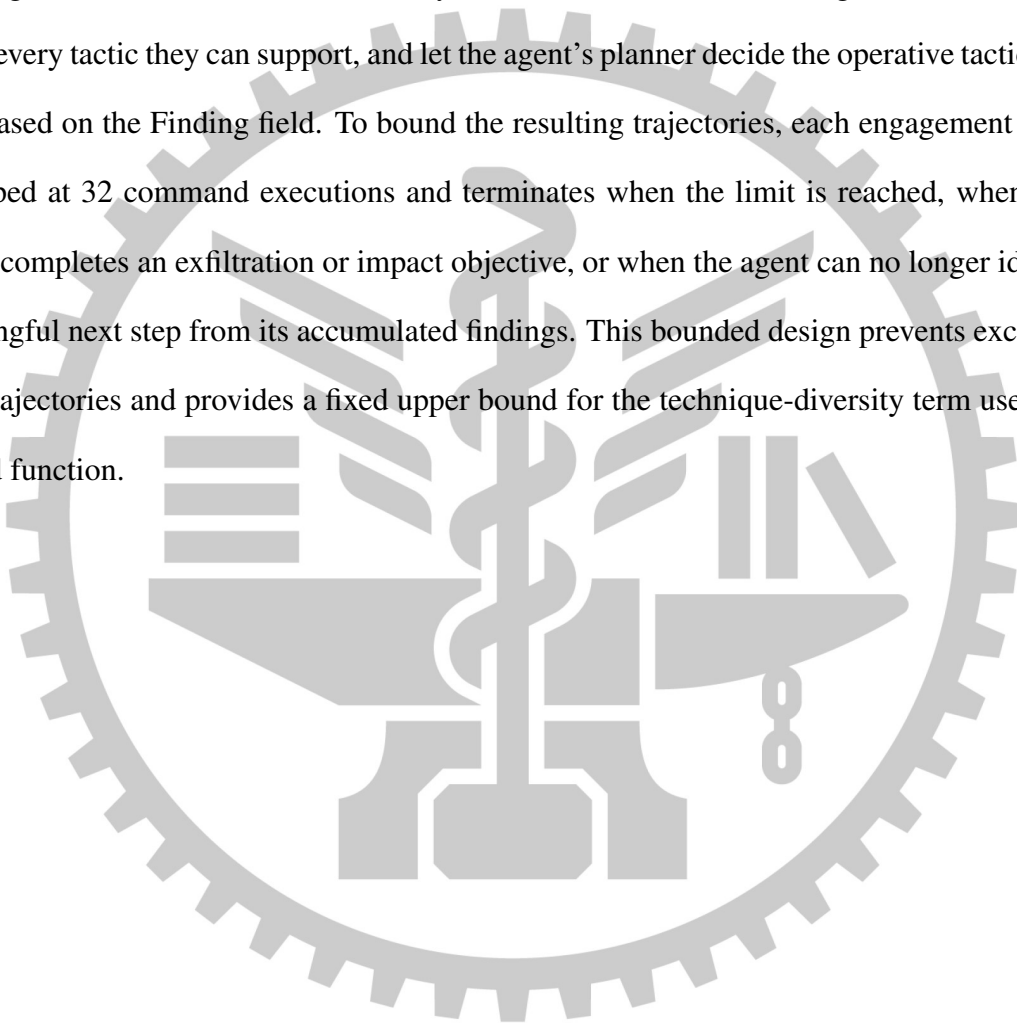
command sequences that reflect evolving adversarial intent, making the resulting trajectories well suited for training and evaluating intent-aware engagement strategies.

As shown in Figure 9, to support this behavior across stateless inference calls, the agent emits a structured output at every interaction step, conditioned on the latest system response. The output consists of three fields: the Finding field records the current attack stage, the active user identity, accessible files, reachable network services, and any obtained credentials, and is updated each round by merging the previous finding with new observations; the Thought field captures the reasoning behind the chosen next step, including the selected tool and its expected outcome; and the tool_params field specifies the concrete tool and arguments to be executed. When selecting a tool, the agent must ground its choice in the information carried in the Finding field, ensuring that every action follows from an understanding of the current environment rather than a predefined script.

To prevent the agent from issuing nonexistent commands or commands with incorrect syntax that would introduce execution errors and distort its decision-making, we provide a curated toolkit in place of free-form command generation. The toolkit is organized according to the seven MITRE ATT&CK tactics supported by our system: Discovery, Collection, Credential Access, Privilege Escalation, Lateral Movement, Exfiltration, and Impact, with each tactic exposing a corresponding set of attack command templates. Specifically, Discovery includes commands such as `whoami`, `id`, `uname -a`, `ls`, `find`, and `ss` for identifying the current user, system information, local files, and reachable services; Collection provides `cat`, `cp`, `sed`, and `tar` for inspecting and packaging local files; Credential Access covers attempts to read sensitive material, including `cat`; Privilege Escalation includes `sudo -l`, enumeration of SUID binaries, and inspection of writable paths; Lateral Movement supports SSH-based connection attempts to discovered hosts or services; Exfiltration provides transfer commands such as `base64`, `curl`, and `scp`; and Impact covers destructive or disruptive operations such as `rm -rf` and `kill`. Several of these templates retain placeholders for arguments such as target paths, hostnames, or process identifiers, which the agent fills in at runtime based on the

contents of the Finding field. This design preserves syntactic correctness while still allowing the agent to adapt each command to the specific state of the environment.

Note that the same command template may appear under multiple tactics, since MITRE ATT&CK classifies behaviors by attacker intent rather than syntactic form. For example, reading `/etc/passwd` is documented under T1087.001 (Discovery) for enumerating local users, and also under T1003.008 (Credential Access) when paired with `/etc/shadow` for offline password cracking. The invocation is identical; only intent differs. We therefore register such commands under every tactic they can support, and let the agent’s planner decide the operative tactic at run-time based on the Finding field. To bound the resulting trajectories, each engagement episode is capped at 32 command executions and terminates when the limit is reached, when the attacker completes an exfiltration or impact objective, or when the agent can no longer identify a meaningful next step from its accumulated findings. This bounded design prevents excessively long trajectories and provides a fixed upper bound for the technique-diversity term used in the reward function.



5.3 SFT Dataset Construction and Training

Table 6: Supervised finetuning configuration

Item	Setting
Base model	LLaMA-3-8B
Training framework	Unsloth
Fine-tuning method	QLoRA-style supervised fine-tuning
Chat template	LLaMa-3-8B
Max sequence length	2048
Base model loading	4-bit
Trainable parameters	LoRA adapter only
Precision	bfloat16
LoRA rank / alpha	16 / 32
Training batch size	2
Gradient accumulation steps	4
Learning rate	2.0×10^{-4}
Epochs	3
Optimizer	AdamW 8-bit
Learning-rate scheduler	Linear
Warmup ratio / weight decay	0.05 / 0.001
Random seed	3407
Best-model selection	Lowest validation loss
Dataset split	463 train / 59 validation / 81 test
Number of raw trajectories	144
Number of SFT samples	Augmented to 603 samples
Multi-label threshold or parsing rule	Set of tactics

After the online engagement phase, we collect 144 raw trajectories, each containing on average 32 commands, since no early stopping mechanism is applied during the simulated interaction and the attacker is allowed to continue acting even after achieving its objective until the 32nd turn. Each trajectory is then segmented into 8-command training samples, where every 8 attacker turns correspond to one defender strategy decision. Each engagement record yields 4 samples, resulting in 576 samples in total. We perform a stratified split that reserves 59 samples for validation and 81 samples for testing. To encourage a more balanced strategy distribution after SFT, we further apply data augmentation on the underrepresented strategy, bringing the final dataset to 603 SFT samples.

The frozen trajectories are then passed to a third-party annotator LLM. The ground truth tactics at each step are anchored by the red team agent’s explicit tool selections, so the an-

notator’s role is to translate these grounded decisions, together with the surrounding command sequence and recorded thought process, into a supervised training sample. Since each 8-command segment may invoke multiple tactics through the attacker’s tool selections, the ground truth tactics for each sample are represented as the set of unique tactics observed in that segment. This set composition reflects the temporal structure of typical engagements: early segments are dominated by discovery with other tactics playing a supporting role, mid-stage segments frequently center on credential access or privilege escalation, while late-stage segments are dominated by collection, accompanied by exfiltration and impact. Tactic prediction is therefore evaluated as a multi-label classification task. For each 8-command window, the predicted tactic set is compared with the ground truth tactic set, and precision, recall, and F1-score are computed over tactic labels rather than over individual samples.

Each labeled sample contains four fields, namely the inferred intent, the ground truth tactics, the deception strategy, and the defender tool parameters. Among these, the inferred intent is the most important field, since the remaining three fields are all derived from it during the strategist’s reasoning process. The hyperparameters used during training are summarized in Table 6. We fine-tune LLaMa-3.1-8B with QLoRA using the open-source Unsloth library.

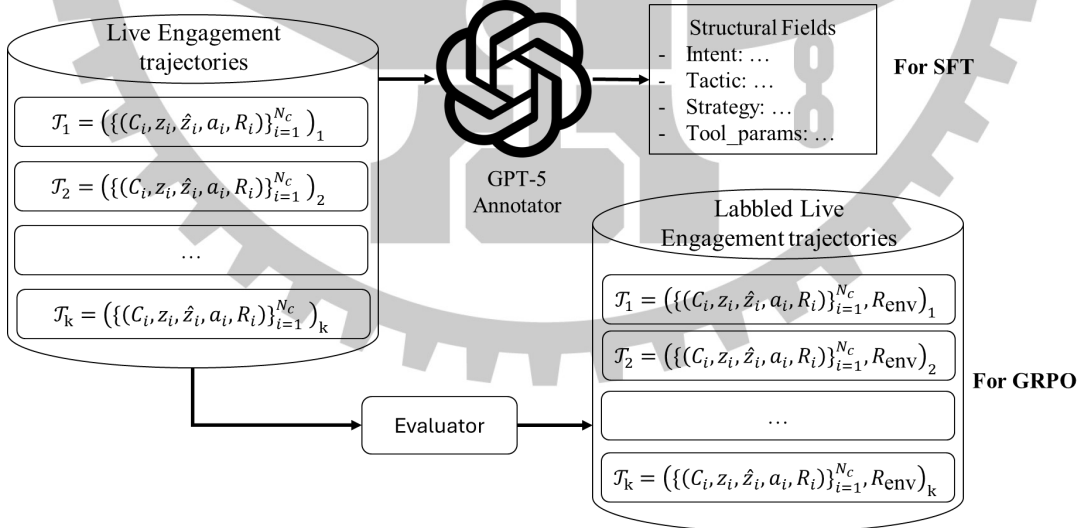


Figure 10: SFT and RFT Training pairs

5.4 Reward Modeling and GRPO Training

For both reward model training and strategist reinforcement fine-tuning, we reuse the same fixed set of engagement trajectories from the SFT stage, but apply a different annotation layer. Whereas SFT consumes data produced by the annotator LLM, this stage relies on the terminal reward computed by the evaluator from the attacker’s tactic depth and technique diversity, as defined in Equation 4.1. The training procedure first fits a BERT-based reward model that learns the mapping from a command sequence and a candidate strategy to the expected reward, and then uses this reward model as a surrogate signal to drive GRPO optimization of the strategist LLM.

Table 7: Reward model configuration

Item	Setting
Base model	google-bert/bert-base-uncased
Training framework	TRL
Training batch size	16
Learning rate	5.0×10^{-6}
Epochs	3
Optimizer	AdamW 8-bit
Learning-rate scheduler	Linear
weight decay	0.01
warmup step	10
warmup ratio	0.05
Random seed	42
Best-model selection	Lowest validation MSE
Dataset split	348 train / 42 validation / 42 test

We adopt BERT as the backbone of our reward model because reward prediction is a fixed-length scoring task rather than a generation task, which aligns more naturally with a bidirectional encoder than with a decoder-only LLM. Compared with recent regression-style reward models trained on top of large language models for open-ended dialogue [24], our setting involves structured and bounded inputs, namely a sequence of shell commands paired with a candidate strategy label, which makes a compact encoder both sufficient in capacity and substantially cheaper to query inside the GRPO loop. The reward model takes as input the concatenation of the command sequence s and a candidate strategy a , and predicts a scalar

reward

$$R_{\text{BERT}}(s, a) = W^\top f_\theta(s \oplus a), \quad (5.1)$$

where $f_\theta(\cdot)$ denotes the pooled [CLS] representation and W is a learned linear head. Conceptually, this plays a role analogous to a Q-function in offline reinforcement learning: the model learns to estimate the expected return of a state, action pair from logged trajectories. Because the environmental reward is only observed at the trajectory level, every state, action pair extracted from the same trajectory inherits the same terminal reward, which introduces credit-assignment noise. However, each strategy is exercised in a wide range of command contexts across the dataset, so consistently effective strategy, context associations are reinforced across trajectories while spurious ones are averaged out, and the reward model generalizes to unseen (s, a) combinations encountered during GRPO training. We optimize the model with a mean squared error objective

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, R_{\text{env}}) \sim \mathcal{D}} [(R_{\text{BERT}}(s, a; \theta) - R_{\text{env}})^2], \quad (5.2)$$

following the regression-style reward modeling paradigm [24]. The corresponding hyperparameters are listed in Table 7. After training, the reward model is expected to produce a discriminative reward distribution that separates effective strategies from ineffective ones for any given command context.

Table 8: Reinforcement finetuning configuration

Item	Setting
Base model	LLaMA-3-8B (SFT)
Training framework	Unsloth
Fine-tuning method	QLoRA-style Reinforcement Fine-tuning
Chat template	LLaMa-3-8B
Max sequence length	2048
Base model loading	4-bit
Trainable parameters	LoRA adapter only
Precision	bfloat16
LoRA rank / alpha	16 / 32
Training batch size	8
Gradient accumulation steps	4
Learning rate	2.0×10^{-5}
Steps	640
Optimizer	AdamW 8-bit
Learning-rate scheduler	Linear
Warmup ratio / weight decay	0.05 / 0.001
Rollout sampler	vLLM
Sampling min-p / top-k	0.01 / 50
Random seed	3407
Best-model selection	Latest Trained Model
Dataset split	144 offline trajectory

Once the reward model converges, we proceed to GRPO training of the strategist LLM, with the hyperparameters summarized in Table 8. The LoRA configuration is kept identical to the SFT stage so that the same parameter subspace is being optimized in both phases. At each training step, the strategist generates 8 candidate responses for a given command sequence, and the BERT reward model scores all 8 to produce the group-relative advantage that drives the policy update. The action space is restricted to a predefined set of 5 primitive operations; outputs that fall outside this set or trigger a parse error are directly assigned a reward of -1 . This penalty encourages the policy to emit tool selections that can be correctly parsed by the MCP toolkit and prevents the post-training model from producing illegal or non-executable instructions. We additionally retain a frozen reference model and add a KL-divergence penalty to the objective, which constrains the updated policy from drifting too far from the SFT initialization. This regularization mitigates reward hacking and allows training to converge more stably toward the strategy preferences encoded by the reward model.

Chapter 6. Experiment Results

6.1 Evaluation setup

Table 9: Baselines in experiments. Llama3.1 refers to Llama-3.1-8B-Instruct.

Metric	Baseline	Description
Attack Depth & Diversity	QRASSH	RL-driven Cowrie honeypot
	LLM Honeypot	LLM-simulated interactive shell
	Prompt-based Strategist	Llama3.1, zero-shot prompting
	SFT Strategist	Llama3.1, SFT on intent dataset
	SFT+GRPO Strategist (Ours)	Llama3.1, SFT then GRPO
Intent Detection Rate	Wazuh	Rule-based SIEM with tactic rules
	SecBERT	SecBERT, SFT on intent dataset
	Prompt-based Strategist	Llama3.1, zero-shot classification
	SFT Strategist	Llama3.1, SFT on intent dataset
Intent-Strategy Alignment	Prompt-based Strategist	Llama3.1, prompting only
	SFT Strategist	Llama3.1, SFT on intent dataset

In this chapter, we evaluate our proposed framework from three complementary perspectives, each targeting a distinct aspect of the system. First, we assess the overall Engagement Performance of the honeypot through interactions with attackers, using metrics commonly adopted in honeypot research. Second, we examine the strategist component by measuring the Intent Detection Rate and Intent-Strategy Alignment, which together validate whether the strategist can correctly infer attacker intent and produce strategies and tool selections aligned with the ground-truth intent. Finally, we analyze the training effectiveness of the Reward Model and GRPO, which underpin the learning pipeline of our framework. Table 9 summarizes the baselines and metrics used in each of these three evaluation dimensions.

For the engagement experiments, Attack Depth and Diversity are evaluated through live interactions with attacker. The attacker is driven by Gemini, which differs from the LLM used during training, while the prompt remains identical to ensure consistency. An attacker is allowed to leave the environment either upon achieving its objective or when it becomes suspicious of the system. Each engagement permits a maximum of 32 interactions, and we conduct a total of 50 engagement sessions. The environment is reset to the same initial state at the beginning of every engagement, which prevents the attacker from exploiting historical records to replicate prior attack paths.

The Intent Detection Rate is measured on the dataset used during SFT, while the Intent-Strategy alignment Score is derived from the semantic relevance between the strategies inferred from 30 command sequences and their annotated intents. These command sequences are deliberately designed to cover early, mid, and late-stage attack characteristics, ensuring that the strategist is evaluated across the full attack lifecycle.

6.2 Engagement performance

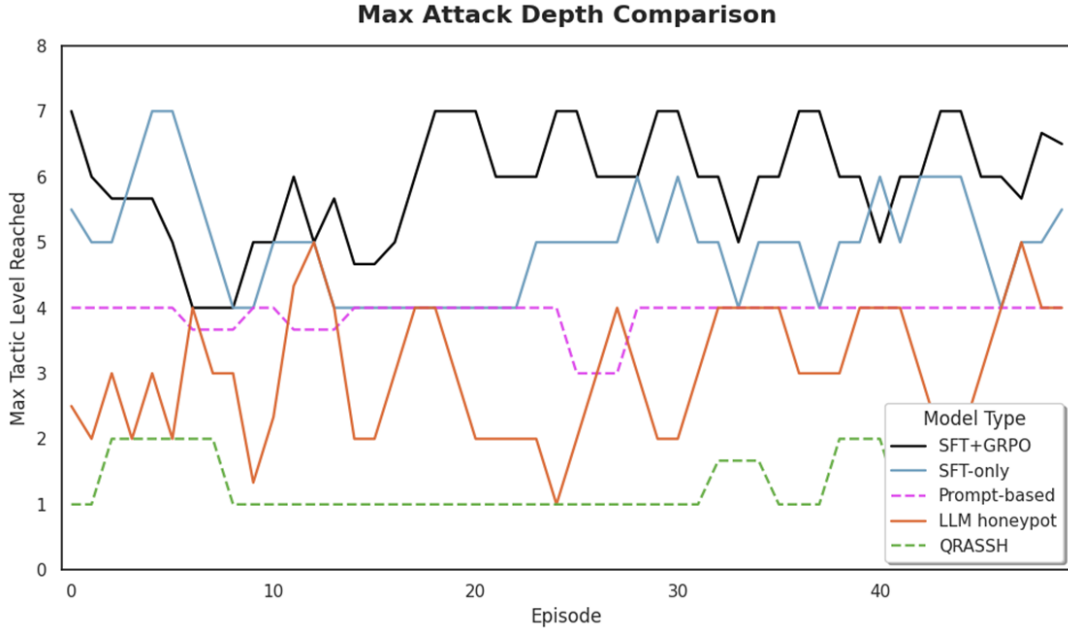


Figure 11: Max attack depth over 50 engagement sessions.

We evaluate engagement performance by deploying the same attacker against five adaptive honeypot systems: QRASSH, LLM Honeypot, Prompt-based strategist, SFT-only strategist, and SFT+GRPO strategist. Each engagement is annotated with tactics and techniques, following the same procedure used in training. Attack Depth is the maximum reached stage taken as the depth of that engagement. Attack Diversity is computed as the cardinality of the set of techniques observed in one session.

As shown in Figure 11, the Attack Depth results reveal a clear separation between the strategist-based systems and the two baselines. SFT+GRPO achieves the highest average max depth of 5.92, followed by SFT-only at 4.96 and Prompt-based at 3.90, while LLM Honeypot and QRASSH trail at 3.00 and 1.32 respectively. The strategist-based systems consistently outperform the baselines because they actively steer the attacker toward later stages, whereas QRASSH and LLM Honeypot are bottlenecked by environment fidelity and fail to retain attackers long enough for strategy to matter. Within the strategist group, the ordering reflects the progressive benefit of each training stage: Prompt-based lacks adversary understanding and defaults to conservative responses, SFT-only acquires more accurate intent awareness but occasionally commits to suboptimal strategies, and SFT+GRPO further refines its choices through reward-driven optimization.

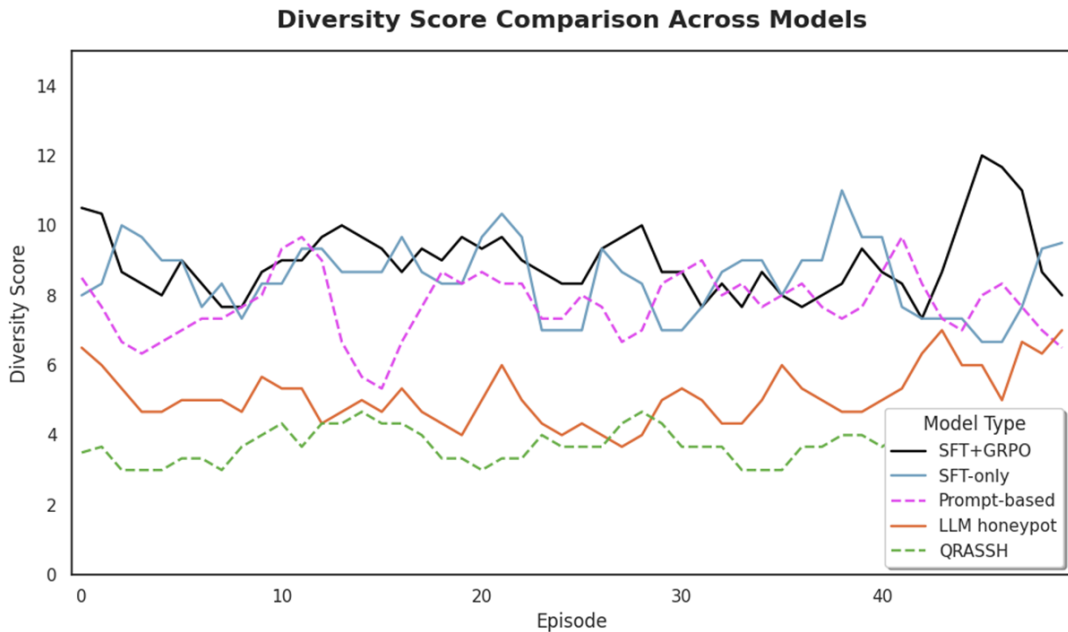


Figure 12: Unique tactic counts over 50 engagement sessions.

Figure 12 shows that Attack Diversity follows the same ordering but with substantially smaller gaps among the top three: SFT+GRPO reaches 8.96, SFT-only 8.48, Prompt-based 7.74, LLM Honeybot 5.10, and QRASSH 3.66. The compressed gap among strategist-based systems reflects the nature of system exploration itself: the set of techniques an attacker can apply when probing a Linux environment is largely shared across engagements, so systems that sustain attackers to similar depths naturally surface overlapping technique sets. The larger drop to LLM Honeybot and QRASSH instead reflects their inability to guide attackers beyond early stages, where the applicable technique pool is inherently narrower. The reward-driven refinement in SFT+GRPO still provides a consistent additional gain over SFT-only across both depth and diversity, confirming the value of optimization on top of intent-aware supervision.

6.3 Intent Detection and Intent-Strategy Alignment

The Intent Detection benchmark evaluates whether the strategist can correctly identify attacker tactics from observed commands. We compare four detectors spanning different paradigms: Wazuh as a rule-based detector built on public rulesets, SecBERT as a learning-based classifier fine-tuned on the same dataset used by our SFT pipeline, the Prompt-based Strategist, and the SFT-only Strategist. Each test sample consists of eight attacker commands paired with their ground-truth tactics, and detection performance is reported in terms of F1 score, precision, and recall.

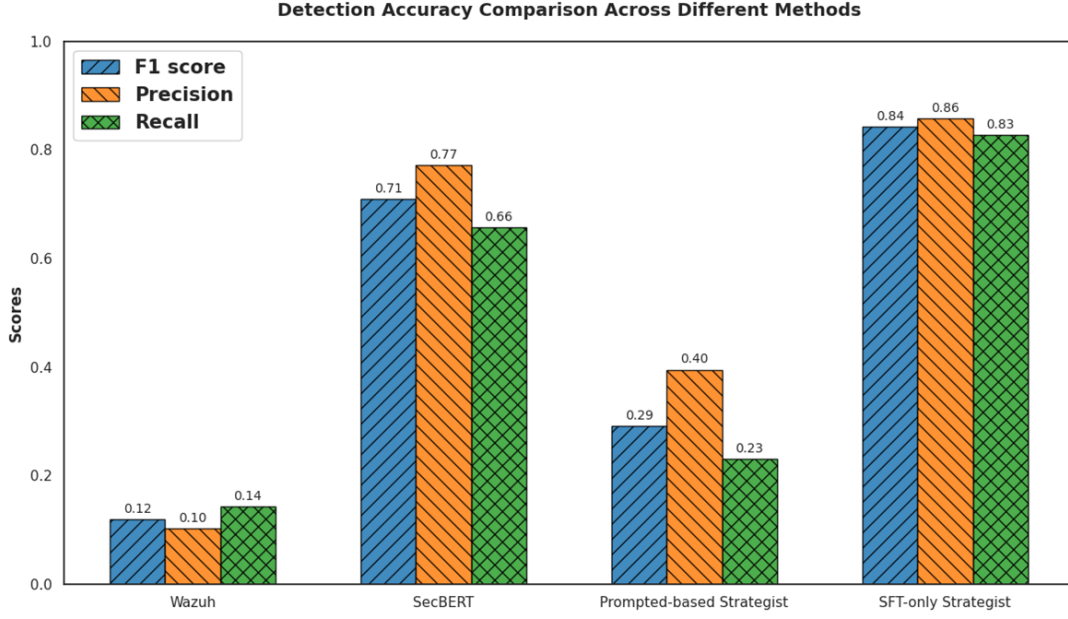


Figure 13: Detection accuracy comparison across different methods.

As shown in Figure 13, the SFT-only Strategist achieves the highest F1 score of 0.84, substantially outperforming SecBERT at 0.71, the Prompt-based Strategist at 0.29, and Wazuh at 0.12. The gap between the SFT-only Strategist and the Prompt-based Strategist is the cleanest indicator of fine-tuning’s contribution, since the two share the same model and inference procedure and differ only in whether the model has been fine-tuned on our annotated dataset. The further gap between the SFT-only Strategist and SecBERT, both fine-tuned on the same data, likely reflects the larger model capacity and broader pretraining of the LLM-based strategist compared to the BERT-based classifier. Wazuh performs worst because rule-based matching relies on fixed signatures and cannot generalize to command variations or compositions that fall outside its predefined patterns. We note that while SecBERT alone could in principle serve as an intent classifier, our framework requires a strategist that not only identifies attacker tactics but also produces a coherent engagement strategy and deployment plan in the same forward pass. An LLM-based strategist is the natural choice for this unified role, and the results here confirm that it does so without sacrificing detection accuracy.

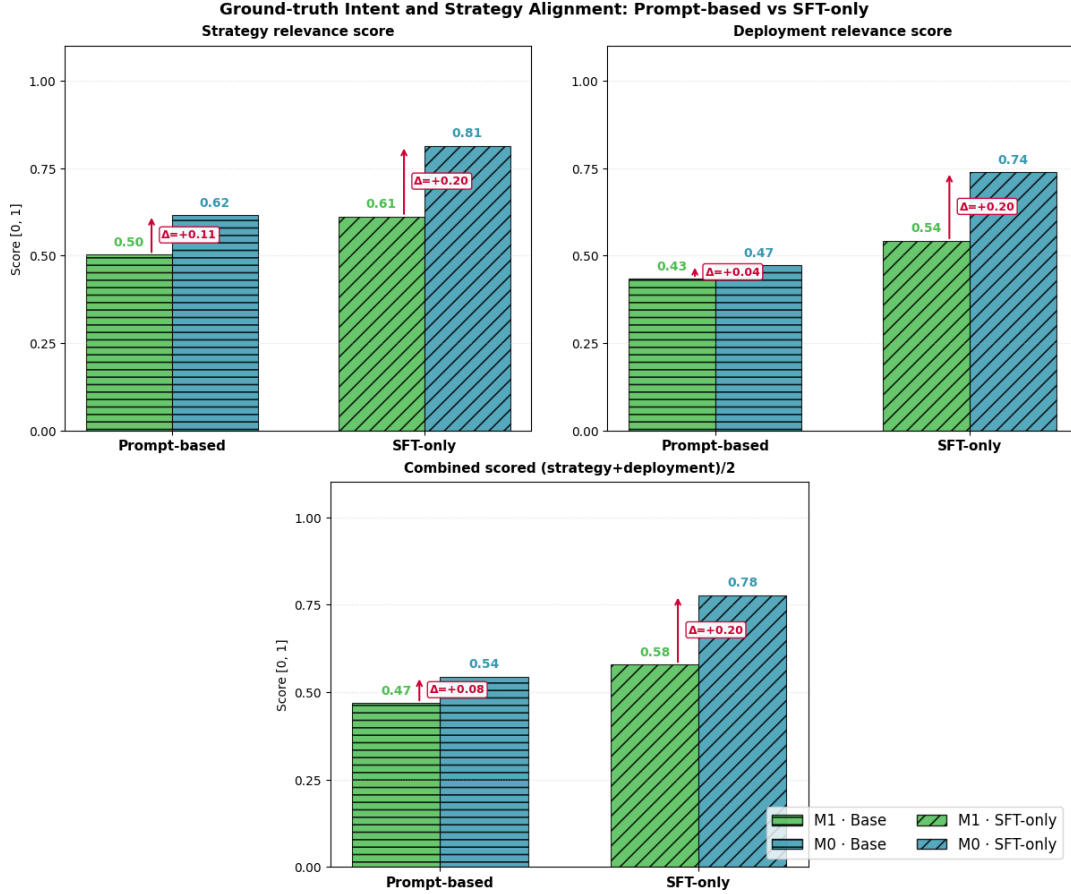


Figure 14: Strategy and deployment relevance under intent-conditioned (M0) and intent-free (M1) generation, for both base and fine-tuned models.

The Strategy Alignment scores benchmark evaluates whether the strategist produces strategies and deployments that are consistent with the attacker’s intent inferred from observed commands. The dataset consists of 30 command sequences spanning early-, mid-, and late-stage characteristics of attacker progression, each annotated with a ground-truth intent, and are independently annotated from the data used during SFT. For each sample, the strategist outputs 2 artifacts that are jointly scored: a natural-language **strategy** describing what to deploy and why, and **tool_params** specifying the deployment tool and its parameters. Evaluation is performed by an LLM judge (GPT-5) using a fixed rubric and produces two scores in $[0, 1]$.

We compare four configurations along two axes: Prompt-based vs. SFT-only and with intent and without intent inference. **Strategy Relevance** measures the consistency between the proposed strategy and two reference signals: (1) the specific commands observed in the session, and (2) the ground-truth attacker intent. The first criterion is the more demanding of

the two: a high-scoring strategy must explicitly reference the binaries, file paths, and action verbs that appear in the command sequence, and articulate the engagement loop in concrete terms, namely what deception is placed, what the attacker is expected to do with it. Strategies that describe the deception in abstract terms, for example proposing to “place a token for the attacker to retrieve” without naming what the attacker has actually been doing, fail this criterion regardless of how plausible the prose reads in isolation. **Deployment Relevance** treats the (tool_id, tool_params) pair as a single deployment plan and evaluates whether this plan is consistent with the attacker’s intent and the specific techniques observed in the commands. A format constraint is also enforced: each parameter must hold a single value within the predefined parameter space, and any malformed or out-of-space parameter caps the score regardless of the rest of the deployment. This ensures that we are not rewarding deployments that look reasonable in prose but cannot actually be instantiated within our system.

As shown in Figure 14, conditioning on inferred intent improves relevance for both models: the Prompt-based score rises from 0.47 to 0.54, while the SFT-only score rises from 0.58 to 0.78. Two observations follow. First, explicit intent inference is a useful signal for strategy generation across both models. Second, SFT amplifies its effect, since the fine-tuned model is better able to translate accurate intent into strategies and deployments that match what the situation demands. This stronger alignment with attacker intent is what ultimately translates into the deeper and more diverse engagements observed in Section 6.2.

6.4 Reward Model and GRPO Training

The previous sections established that SFT+GRPO outperforms SFT-only on both engagement metrics. This section examines the training pipeline that produces this gain, with two questions in mind: whether the reward model learns a meaningful preference signal over strategies, and whether GRPO successfully translates that signal into policy behavior. Together they verify that the two-stage pipeline is sound from an RL perspective, with the reward model providing a stable target and GRPO acting on that target to reshape the strategist’s policy.

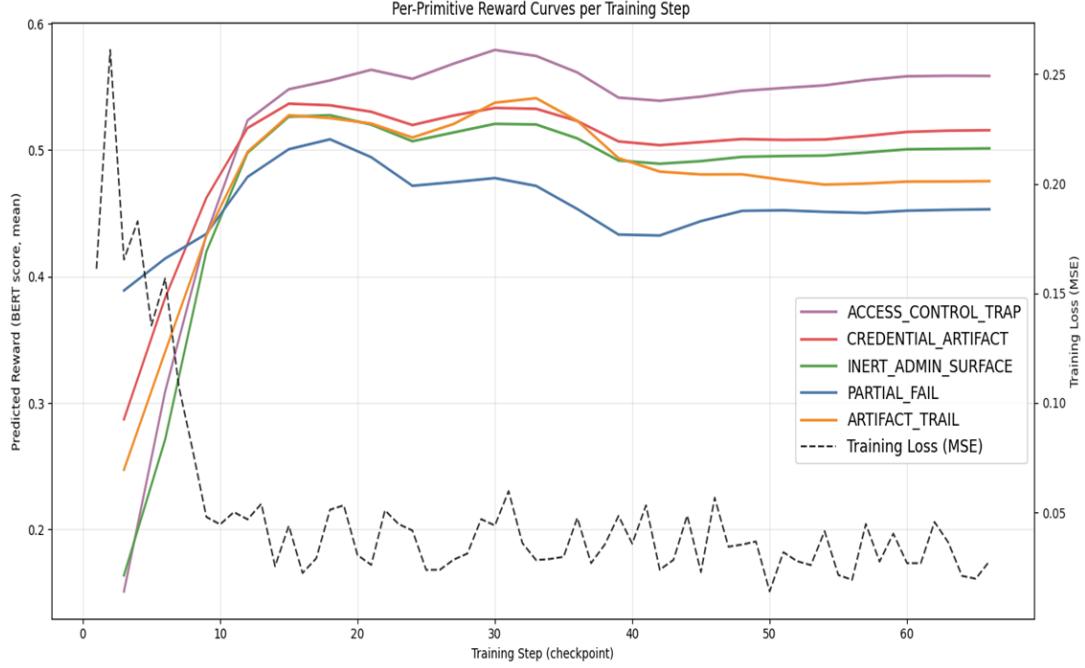


Figure 15: Per-primitive predicted reward and training loss across training checkpoints.

Figure 15 reports the per-primitive predicted reward and the training loss across checkpoints. The loss converges around step 30 and remains stable thereafter, indicating that the model fits the (command sequence, strategy primitive) to outcome mapping without overfitting or oscillation. More importantly, the predicted rewards for each primitive, averaged across command sequences, spread out and settle into a consistent ordering with clear separation between the highest- and lowest-ranked primitives. The fact that this ordering emerges and persists across checkpoints, rather than the rewards collapsing to a single value, indicates that the reward model has extracted a usable preference signal over strategies from the offline data. The absolute reward values are less important than this relative ordering, since GRPO consumes reward as a group-relative signal rather than an absolute target. A consistent and well-separated ordering is precisely what GRPO needs to perform reliable credit assignment during policy updates, because credit is assigned by comparing rewards within a sampled group rather than against an external reference.

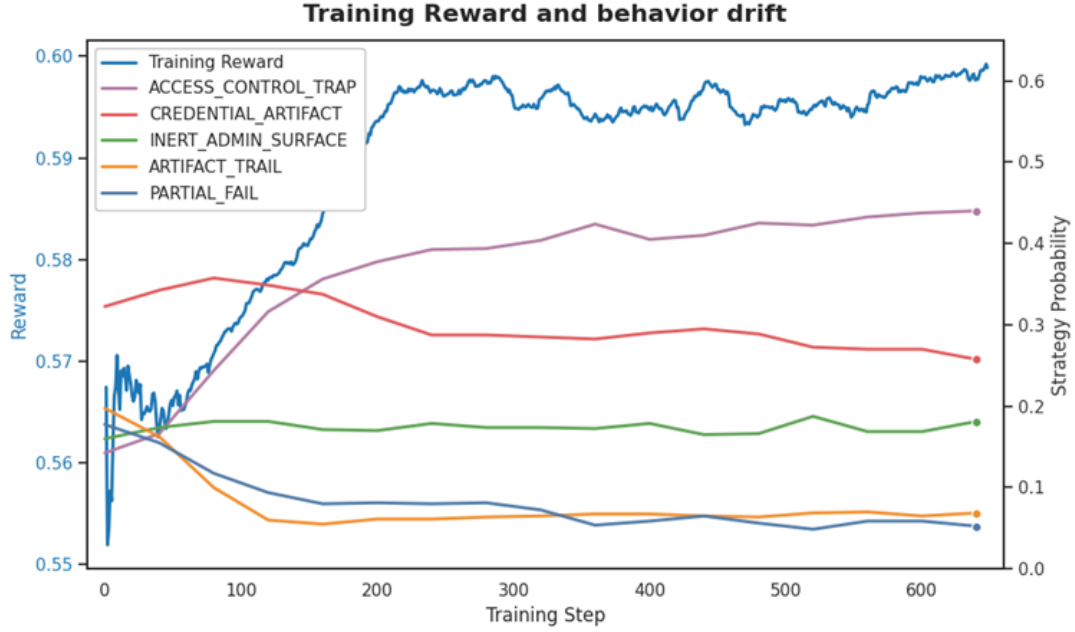


Figure 16: Training reward and strategy probability drift during GRPO training.

The strategist’s behavior during GRPO training, illustrated in Figure 16, shows that the training reward rises steadily throughout optimization, and the policy distribution drifts in a direction consistent with the reward model’s ordering: `ACCESS_CONTROL_TRAP` and `CREDENTIAL_ARTIFACT`, the two primitives ranked highest by the reward model, gradually increase in selection frequency, while the lower-ranked primitives decrease. The drift is gradual rather than abrupt, which is consistent with GRPO’s group-relative update mechanism: the policy moves toward higher-reward primitives proportionally to the reward gap observed within each sampled group, so primitives with a clear reward advantage are reinforced first while smaller gaps produce smaller updates. This alignment between policy drift and reward model ordering confirms that the proxy reward is informative enough to guide policy optimization, and that the two-stage pipeline behaves as designed: SFT first aligns the strategist to the annotated dataset, after which GRPO shifts its policy toward the primitives that the reward model identifies as most effective in our honeypot environment.

6.5 Discussion and Limitation

A consistent pattern emerges across this chapter’s evaluations. QRASSH and LLM HoneyPot lose attackers early because their responses contain inconsistencies that attackers quickly recognize. Strategist-driven systems avoid this by actively shaping the attacker’s experience, but only when they reason about attacker intent. The Prompt-based Strategist illustrates this limit: it shares the same model and inference path as the SFT-only Strategist, yet without fine-tuning on annotated intent it produces strategies that read as plausible but do not engage what the attacker is actually doing. GRPO then closes a second gap. Knowing the attacker’s intent tells the strategist what situation it is in, but not which primitive will have the largest effect in that situation, and this is what reward-guided refinement provides.

These conclusions should be read with the experimental scope in mind. Our evaluation relies entirely on LLM-based attackers, both during training and at evaluation time. Although we switch to a different attacker model for evaluation to test robustness, this only covers variation within the LLM family, and the strategist’s preferences may not carry over to human attackers, who can be more patient or more skeptical of the artifacts our primitives plant. A further consideration concerns how the strategist acts on the environment: each primitive influences the engagement only indirectly, since the attacker must encounter the placed artifact through their own exploration before it has any effect. This indirection is partly by design, since it keeps the strategist’s behavior aligned with how a realistic system would appear rather than reacting visibly to each command, but it does mean the timing and reach of any primitive depend on what the attacker chooses to investigate.

Chapter 7. Conclusions and Future works

7.1 Conclusions

This thesis presented **IntenSensePot**, an intent-aware honeypot framework that unifies adversarial intent inference and deceptive strategy selection within a single fine-tuned LLM. Departing from prior RL-based honeypots that treat policy models as either isolated strategists or rigid command-level executors, our work bridges the **perception–decision gap** by aligning a single LLM with both MITRE ATT&CK and MITRE Engage. The framework is realized through a two-stage offline pipeline: SFT instills an explicit Chain-of-Thought that maps noisy command sequences to adversarial intent, while GRPO reshapes the model’s default conversational policy into a domain-specific engagement intuition.

Across three evaluation axes, the proposed approach demonstrates consistent gains. **First**, intent detection F1-score improves substantially over a prompt-based baseline using the same backbone, surpassing both rule-based (Wazuh) and learning-based (SecBERT) detectors. **Second**, ablation studies show that conditioning strategy generation on inferred intent meaningfully raises intent–strategy alignment, and that this alignment is what ultimately translates into stronger live performance. **Third**, in 50 live engagement sessions, IntenSensePot substantially increases both maximum engagement depth and technique diversity compared to QRASSH and LLM Honeypot baselines, while training-time analysis confirms that GRPO faithfully shifts the policy distribution toward higher-reward primitives.

Taken together, these results support three broader claims. **(i)** Intent awareness is a prerequisite, not an optional enhancement, for adaptive engagement: without it, even a capable LLM defaults to plausible-sounding but operationally hollow strategies. **(ii)** Honeypot-specific be-

havior cannot be obtained from prompting alone, because pre-trained alignment systematically biases LLMs away from deception; fine-tuning is necessary to reclaim this design space. (iii) Decoupling abstract engagement strategy from low-level execution via an MCP toolkit makes the learned policy portable across backends, addressing a long-standing limitation of system-bound RL honeypots.

These conclusions should be read with the experimental scope in mind. Both training and evaluation rely on LLM-driven attackers, and while we switch model families between the two phases, this variation only covers behavior within the LLM family; human attackers may be more patient or more skeptical of planted artifacts. A second consideration is that each engagement primitive influences a session only after the attacker independently encounters the deployed artifact, meaning primitive effectiveness depends partly on attacker exploration patterns rather than strategist quality alone.

7.2 Future Works

Several directions naturally extend this work. The most immediate priority is to validate IntenSensePot against human red-team operators, which would test whether the strategy preferences learned from LLM attackers transfer to adversaries with different patience profiles and detection heuristics. Closely related, incorporating attackers with heterogeneous skill levels at training time could broaden the strategist’s exposure beyond the behavioral patterns of a single LLM family.

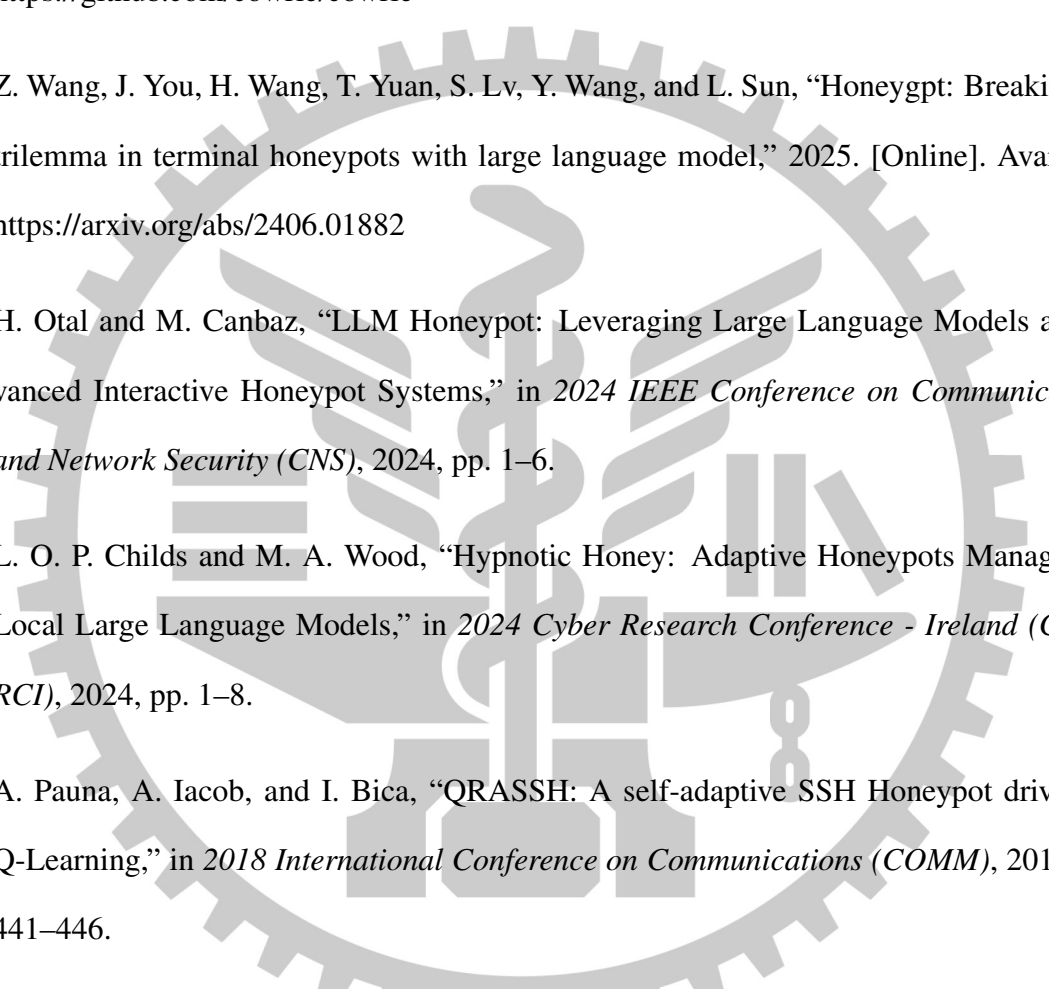
A second direction concerns the expressiveness of the action space. Expanding beyond the current five primitives, for instance through compositional strategies that chain multiple Engage activities or time-conditioned primitives that escalate as the engagement matures, would let the strategist express more nuanced deception. Relatedly, exploring process-supervised reward modeling, where individual strategy decisions receive their own quality signal, could reduce the credit-assignment noise inherent to trajectory-level rewards.

Finally, because the strategist operates over an abstract MCP toolkit rather than concrete

shell commands, the same policy should in principle transfer to substantially different backends such as Windows hosts, Active Directory environments, or industrial control systems. Empirically validating this portability would substantiate the framework's claim of system independence and broaden its practical applicability.



References

- 
- [1] M. Oosterhof, “Cowrie Honeybot,” 2020. [Online]. Available: <https://github.com/cowrie/cowrie>
- [2] Z. Wang, J. You, H. Wang, T. Yuan, S. Lv, Y. Wang, and L. Sun, “Honeygpt: Breaking the trilemma in terminal honeypots with large language model,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.01882>
- [3] H. Otal and M. Canbaz, “LLM Honeybot: Leveraging Large Language Models as Advanced Interactive Honeybot Systems,” in *2024 IEEE Conference on Communications and Network Security (CNS)*, 2024, pp. 1–6.
- [4] L. O. P. Childs and M. A. Wood, “Hypnotic Honey: Adaptive Honeybots Managed by Local Large Language Models,” in *2024 Cyber Research Conference - Ireland (Cyber-RCI)*, 2024, pp. 1–8.
- [5] A. Pauna, A. Iacob, and I. Bica, “QRASSH: A self-adaptive SSH Honeybot driven by Q-Learning,” in *2018 International Conference on Communications (COMM)*, 2019, pp. 441–446.
- [6] S. Dowling, M. Schukat, and E. Barrett, “Using Reinforcement Learning to Conceal Honeybot Functionality,” in *ECML PKDD 2018: Machine Learning and Knowledge Discovery in Databases*. Springer, 2018.
- [7] P. Prasad, N. Girish, S. V, A. Vh, and P. K. S, “Generative AI SSH Honeybot With Reinforcement Learning,” in *2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT)*, 2025, pp. 770–775.

- [8] S. Ren *et al.*, “Adaptive SSH Honeypot for Power Monitoring Systems: A Synergistic Approach Combining Reinforcement Learning and Large Language Models,” in *2025 2nd International Symposium on AI and Cybersecurity (ISAICS)*, 2025, pp. 1–8.
- [9] R.-H. Hwang, J.-C. Huang, Y.-C. Lai, and Y.-D. Lin, “Leveraging Reinforcement Learning for LLM-based Generic Honeypots with MITRE Engage Framework,” *IEEE Transactions (Pending/Published)*, 2025.
- [10] R.-H. Hwang, C. Yu Hsieh, Y.-C. Lai, and Y.-D. Lin, “Active Directory Deception: A Reinforcement Learning Honeypot Aligned with MITRE Engage,” *IEEE Transactions (Pending/Published)*, 2025.
- [11] Y.-D. Lin, R. Mankaev, R.-H. Hwang, Y.-C. Lai, and D. Sudyana, “Reinforcement Learning-driven Honeypots: Tactic-based Defense for Industrial Control Systems,” *IEEE Transactions (Pending/Published)*, 2025.
- [12] “Honeyd: A virtual honeypot daemon.” San Francisco, CA: USENIX Association, Aug. 2002.
- [13] U. Tamminen, g0tmilk, K. Durechova, A. Smith, and gymzombie, “desaster/kippo,” <https://github.com/desaster/kippo>, nov 19 2023. [Online]. Available: <https://github.com/desaster/kippo>
- [14] MITRE, “MITRE ATT&CK: A knowledge base of adversary tactics and techniques based on real-world observations,” 2024. [Online]. Available: <https://attack.mitre.org/>
- [15] Joe, R. Kervella, dependabot[bot], T. M. Ghatas, Raf, rwincey, R. Kervella, J. Golovich, maxlondon, D. Tomaschik, mr.The, Anas, c2biz, C_Sto, A. Iandoli, usiegl00, Marc, A. Flores, dono, C. Smith, hypnoticpattern, kmathisbf, C. Burn, jjensn, guervild, S. Haas, Chase, M. Hamilton, K. Avery, and D. Breuker, “Bishopfox/sliver,” <https://github.com/BishopFox/sliver>, apr 19 2026. [Online]. Available: <https://github.com/BishopFox/sliver>

- [16] Red Canary, “Atomic red team: Small and highly portable detection tests based on MITRE ATT&CK,” GitHub. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team>
- [17] MITRE, “MITRE Engage: A framework for engaging adversaries to gather intelligence and inform defense,” 2024. [Online]. Available: <https://engage.mitre.org/>
- [18] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2203.02155>
- [19] D. Campbell, N. Kale, U. M. Schwag, B. Herring, C. Q. Knight, D. Borges, N. Price, and A. Levinson, “Defensive refusal bias: How safety alignment fails cyber defenders,” in *ICLR 2026 Workshop*, 2026, published as a workshop paper at ICLR 2026. [Online]. Available: <https://arxiv.org/abs/2603.01246>
- [20] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, and P. Henderson, “Fine-tuning aligned language models compromises safety, even when users do not intend to!” in *International Conference on Learning Representations (ICLR)*, 2024.
- [21] Anthropic, “Model Context Protocol (MCP),” Anthropic Documentation, 2024. [Online]. Available: <https://docs.anthropic.com/en/docs/claude-code/mcp>
- [22] M. Var Naseri, I. Welch, J. Haseeb, and M. Mansoori, “Q-cowrie: An adaptive honeypot to analyse attackers ’ behaviour,” *Int. J. Inf. Secur.*, vol. 25, no. 2, Feb. 2026. [Online]. Available: <https://doi.org/10.1007/s10207-026-01221-5>
- [23] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins, “Solving math word problems with process- and outcome-based feedback,” *arXiv preprint arXiv:2211.14275*, 2022.

- [24] Z. Wang, A. Bukharin, O. Delalleau, D. Egert, G. Shen, J. Zeng, O. Kuchaiev, and Y. Dong, “HelpSteer2-Preference: Complementing ratings with preferences,” *arXiv preprint arXiv:2410.01257*, 2024.



Appendix 附錄

本附錄整理撰寫論文時常見的排版與語法注意事項，供大家撰寫與校稿時參考，請記得看 latex 的語法。(此段落主要由 ChatGPT 撰寫整理 XD)

文字與標點

- 英文引號請使用成對反引號與直引號：“text” “text”。
- 破折號（dash）有三種：‘-’（連字號，如 *pre-trained*）、‘—’（範圍，如 10–20）、‘—’（句中強調，如 the result—as expected—was correct）。
- 省略號請使用 `\ldots` 而非三個句點：A, B, C..., Z。
- 數值與單位間需留不換行空白：10 km，30°C。
- 中英文混排時，中文與英文之間宜留一半形空白，例如：中文 English 中文。

數學與符號

- 數學變數自動為斜體，單位或文字常數須使用 `\mathrm{}` 或 `\text{}`，如 $v = 10 \text{ m/s}$ 。
- 三角函數、極值運算等應以數學命令表示： $\sin \theta$, $\max(x, y)$ 。
- 向量或矩陣可用粗體： $\mathbf{x} = [x_1, x_2]^\top$ 。
- 多行方程式請使用 `align` 環境，勿以多個 `equation` 拆寫。

圖表與引用

- 圖片標題（caption）置於圖下方
- 表格標題置於表上方。
- 參照圖表時請使用 `\ref{}` 或 `\autoref{}`，勿手動輸入編號。
例：As shown in Fig. ??.
- 表格建議使用 `booktabs` 套件：`\toprule`、`\midrule`、`\bottomrule`。

文獻與引用

- 文獻引用使用 `\cite{key}` 或 `\parencite{key}` 。
- 多篇同時引用可寫為 `\cite{a,b,c}` [1-3] 。
- 特殊大小寫（如 GPU, LaTeX）須以大括號保留：`{GPU}` 。

